Software Architecture & Design of Large Scale Systems

By Michael Pogrebinsky



Copyright Notice

The workbook's contents, including (but not limited to) all written material and images, are protected under international copyright and trademark laws.
Any redistribution or reproduction of part or all of the contents in any form is prohibited.
You may not, except with written permission from the author, distribute or commercially exploit the content.
Nor may you transmit it or store it on any other website, forum, or other forms of electronic retrieval systems.
You may print or download to your local hard disk extracts for your personal and non-commercial use only.

© 2025 Top Developer Academy LLC by Michael Pogrebinsky. All rights reserved.

Software Architecture & Design of Large Scale Systems

Introduction to System Requirements & Architectural Drivers **1**
- Introduction to System Design & Architectural Drivers 1
- Feature Requirements - Step-by-Step Process 2
- System Quality Attributes Requirements 3
- System Constraints in Software Architecture 4

Most Important Quality Attributes in Large-Scale Systems **5**
- Performance 5
- Scalability 7
- Availability - Introduction & Measurement 8
- Fault Tolerance & High Availability 9
- SLA, SLO, SLI 11

API Design **12**
- Introduction to API Design for Software Architects 12
- RPC 13
- REST API 15

Large Scale Systems Architectural Building Blocks **16**
- DNS, Load Balancing & GSLB 16
- Message Brokers 18
- API Gateway 19
- Content Delivery Network - CDN 21

Data Storage at Global Scale **22**
- Relational Databases & ACID Transactions 22
- Non-Relational Databases 23
- Techniques to Improve Performance, Availability & Scalability Of Databases 24
- Brewer's (CAP) Theorem 26
- Unstructured Data Storage 27

Software Architecture Patterns **28**
- Multi-Tier Architecture 28
- Microservices Architecture 29
- Event-Driven Architecture 30

Strategies for Processing Infinite Streams of Events **33**
- Introduction to Event-Stream Processing and Tumbling Window Strategy 33
- Session Window Event-Stream Processing Strategy 36

Big Data Architecture Patterns **37**
- Big Data Processing Strategies 37
- Lambda Architecture 38

Software Architecture & System Design Practice **39**
- Design a Highly Scalable Discussion Forum 1 - Requirements & API 39
- Design a Highly Scalable Discussion Forum 2 - Functional Architecture Diagram 40

Design a Highly Scalable Discussion Forum 3 - Final Software Architecture 41
Design an E-Commerce Marketplace Platform 1 - Requirements & Sequence Diagram 42
Design an E-Commerce Marketplace Platform 2 - Functional Diagram 44
Design an E-Commerce Marketplace Platform 3 - Final Software Architecture 45

Introduction to System Requirements & Architectural Drivers

Introduction to System Design & Architectural Drivers

- **Requirements** - Formal description of what we need to build
- **Types of Requirements** - Architectural Drivers
    - Features of the System
        - Functional requirements



    - Quality Attributes
        - Non-Functional requirements
            - Examples:
                - Scalability
                - Availability
                - Reliability
                - Security
                - Performance
        - Dictate the software architecture of our system



    - System Constraints
        - Limitations and boundaries

©Michael Pogrebinsky Top Developer Academy topdeveloperacademy.com

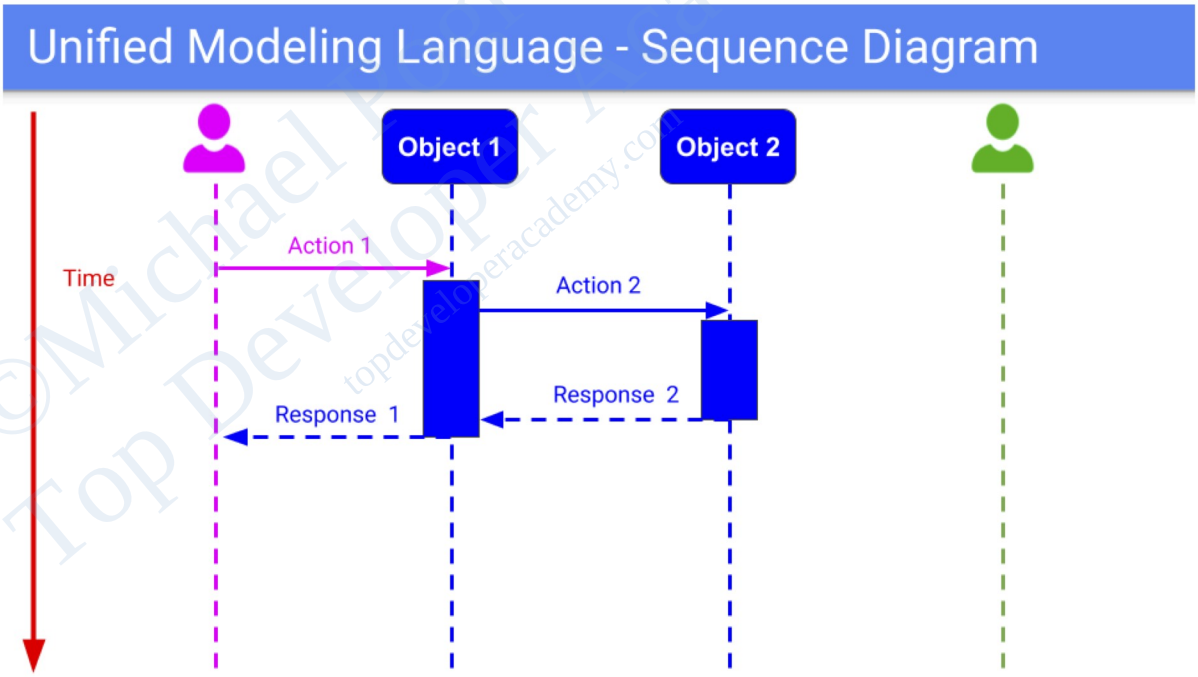
Notes:

There may be a high lvl of ambiguity with the requirements because those may not come from an engineer or a technical person, because of that we need to guide the client with questions like:
- which platform?
- real time vs reservation?
- payment through us or direct payment?

© 2025 Top Developer Academy LLC by Michael Pogrebinsky. All rights reserved.

Feature Requirements - Step-by-Step Process

- Methods of Gathering Requirements
  - Use Cases
    - Situation / Scenario in which our system is used
  - User Flows
    - A Step-By-Step / Graphical representation of each use case
- Requirement Gathering Steps
  - Identify all the actors/users in our system
  - Capture and describe all the possible use cases/ scenarios
  - User Flow - Expand each use case through the flow of events.
    - Each event contains
      - Action
      - Data
- Sequence Diagram
  - The diagram that represents interactions between actors and objects.



©Michael Pogrebinsky Top Developer Academy topdeveloperacademy.com

Notes:

© 2025 Top Developer Academy LLC by Michael Pogrebinsky. All rights reserved.

System Quality Attributes Requirements

- System Quality Attributes
  - Provide a quality measure of how well our system performs on a particular dimension
  - Have a direct correlation with the architecture of our system
- Important Considerations
  - Testability and Measurability
  - Trade-Offs
    - No single software architecture can provide all the quality attributes.
    - Certain quality attributes contradict one another
    - Some combinations of quality attributes are very hard/impossible to achieve
  - Feasibility
    - We need to make sure that the system is capable of delivering with the client asking for

Notes:

Quality attributes are non functional requirements (ex: how fast, how many users, how secure, etc)

They describe:
- the qualities of the functional requirements
- the overall properties of the system

The quality attributes don't measure what the system does, but how well it does on specific dimensions, ex:
- the user needs to pull data *within at most 100 milliseconds*
- the store needs to be available for the clients *at least 99% of the time*

Example of combinations that contradict each other:
> performance - login time < 1 second
> security - username, password, ssl
(this requires more time, so it's impossible to be fast and secure)

!! Important !!
It's good to consult with people who have more expertise on certain areas in order to see if a requirement can actually be fulfilled.

© 2025 Top Developer Academy LLC by Michael Pogrebinsky. All rights reserved.

System Constraints in Software Architecture

- Definition:
  - “A system constraint is essentially a decision that was already either fully or partially made for us, restricting our degrees of freedom.”
- Types of Constraints:
  - Technical constraints
  - Business constraints
    - Forces us to make sacrifices in:
      - Architecture
      - Implementation
  - Regulatory/legal constraints
    - Global
    - Specific to a region
- Considerations:
- We shouldn't take any given constraint lightly
- Use loosely coupled architecture

Notes:

The constraints can actually be good because it is a decision that we don't need to make anymore

Constraints are also called *pillars* since we have to make all the other decisions around them

Business constraints examples:
- the need to use 3rd party shipping or payment
- integration of different 3rd party banks, security and so on

Regulatory constraints examples:
- GDPR

Extra resources:
https://topdeveloperacademy.com/articles/what-exactly-does-a-software-architect-do

https://topdeveloperacademy.com/articles/top-tools-software-architecture-diagrams-software-architects-toolbox

https://topdeveloperacademy.com/articles/togaf-vs-cpsa-f-isaqb-which-software-architecture-certification-is-right-for-you

https://topdeveloperacademy.com/articles/become-a-certified-software-architect-the-benefits-of-cpsa-f-certification

© 2025 Top Developer Academy LLC by Michael Pogrebinsky. All rights reserved.

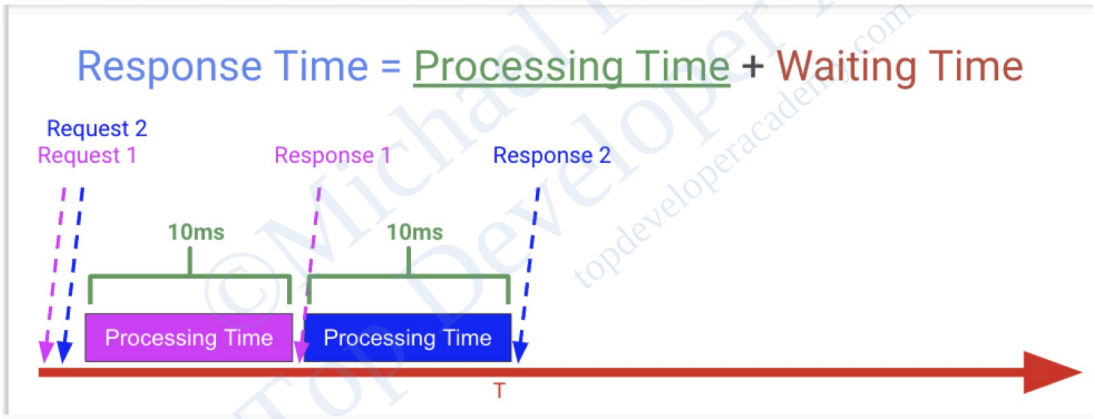
Most Important Quality Attributes in Large-Scale Systems

Performance

- Definitions
  - **Response Time**:
    - The time between a client sending a request and receiving a response
    - Response Time = Processing Time + Waiting Time
    - Waiting Time - Duration of time request/response spends inactively in our system



  - **Throughput**
    - Amount of work performed by our system time
      - Measured in tasks/second
    - Amount of data processed by our system per unit of time
      - Measured in bits/second, Bytes/second, MBytes/second
- Important Considerations:
  - Measuring Response Time Correctly

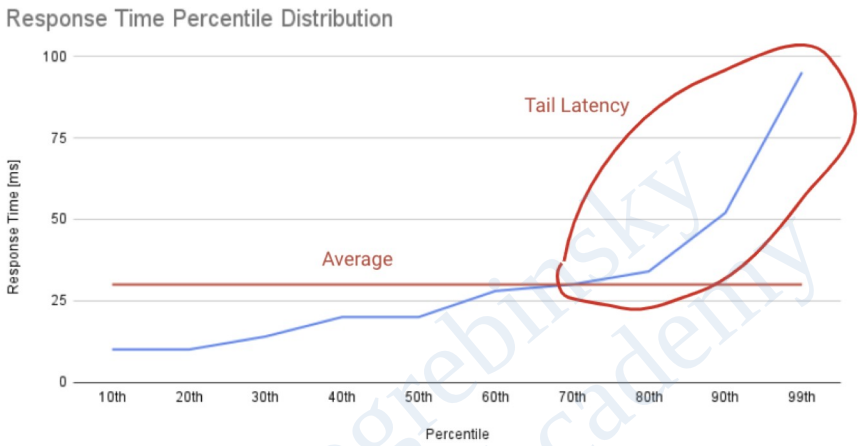


  - Response Time Distribution

©Michael Pogrebinsky Top Developer Academy topdeveloperacademy.com

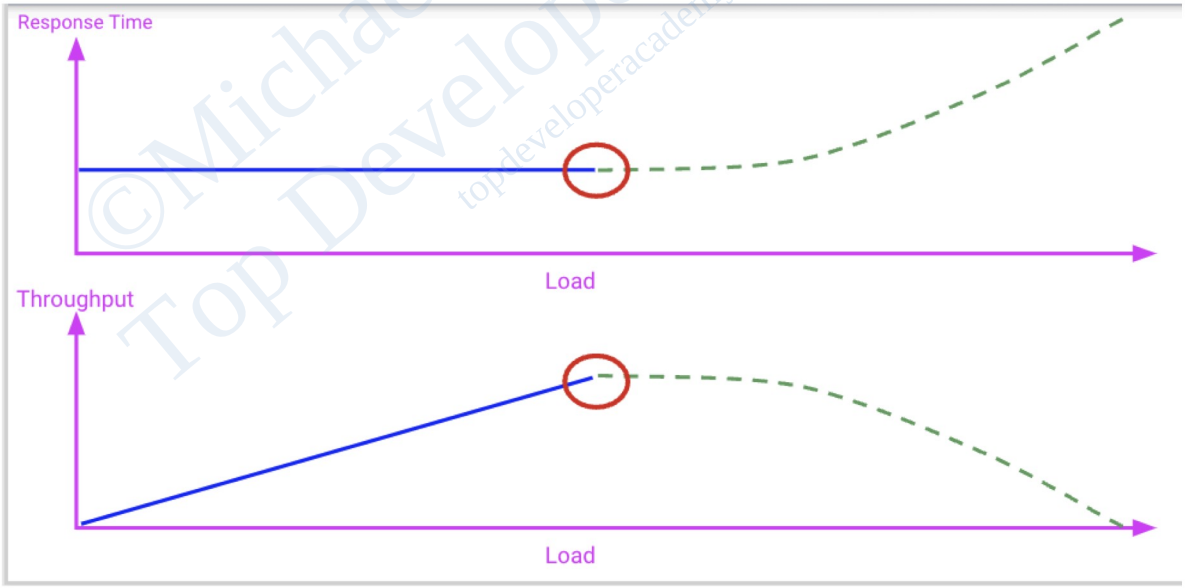
© 2025 Top Developer Academy LLC by Michael Pogrebinsky. All rights reserved.

- **Percentile**: The “xth percentile” is the value below which x% of the values can be found



- **Tail Latency**: The small percentage of response times from a system that take the longest in comparison to the rest of the values

- Performance Degradation

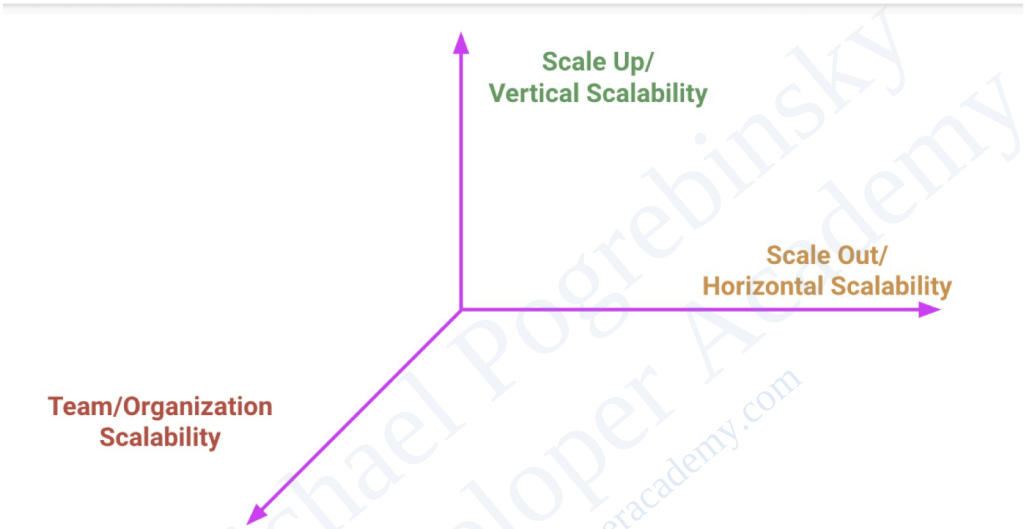


Notes:

© 2025 Top Developer Academy LLC by Michael Pogrebinsky. All rights reserved.

Scalability

- **Scalability Definition:**
  - "The measure of a system's ability to handle a growing amount of work, in an easy and cost-effective way, by adding resources to the system"
- **Types of Scalability**



© Michael Pogrebinsky
Top Developer Academy
topdeveloperacademy.com

  - Vertical Scalability
    - Adding resources or upgrading the existing resources on a single computer
  - Horizontal Scalability
    - Adding more resources in the form of new instances running on different machines
  - Team/Organizational Scalability
    - Software Architecture impacts engineering velocity (team productivity)

Notes:

© 2025 Top Developer Academy LLC by Michael Pogrebinsky. All rights reserved.

Availability - Introduction & Measurement

- **Availability**:
  - *"The fraction of time/probability that our service is operationally functional and accessible to the user."*

> ***Availability = Uptime / (Uptime + Downtime)***

- **Uptime**:
  - The time that our system is operationally functional and accessible to the user
- **Downtime:**
  - The time that our system is unavailable to the user
- **MTTR**
  - Mean Time to Recovery

> ***Availability = MTBF / (MTBF + MTTR)***

©Michael Pogrebinsky Top Developer Academy topdeveloperacademy.com

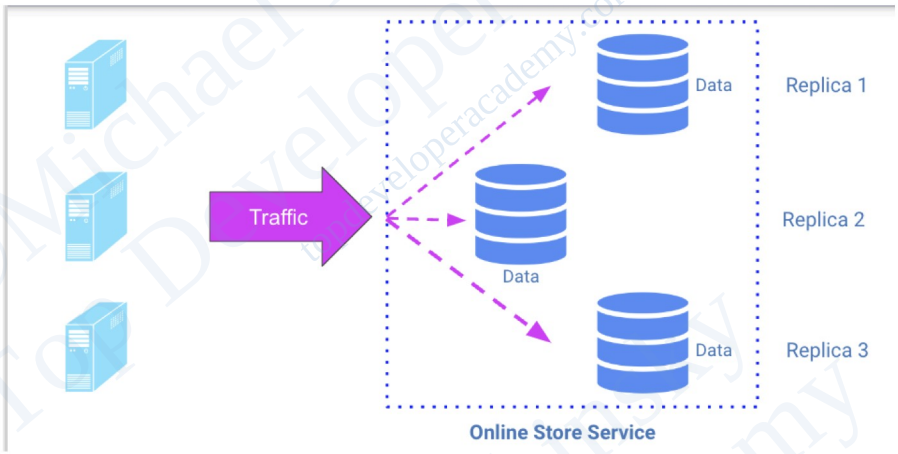
Notes:

MTBF - mean time between failures

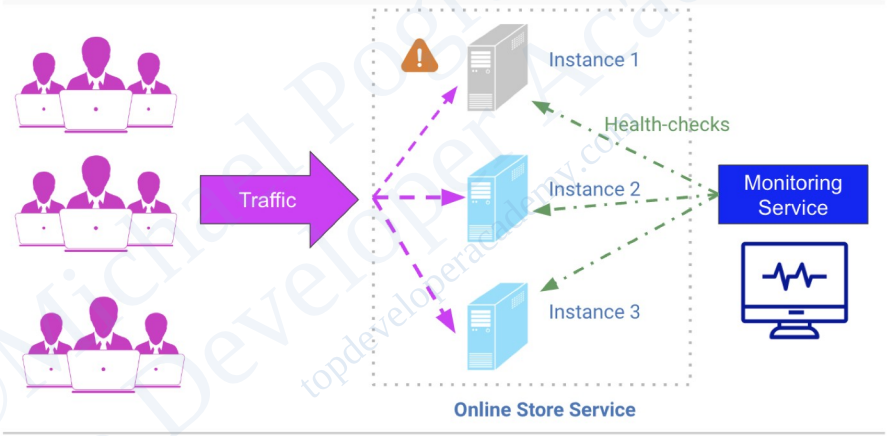
© 2025 Top Developer Academy LLC by Michael Pogrebinsky. All rights reserved.

Fault Tolerance & High Availability

- **Sources of Failure:**
  - Human Error
  - Software Errors
  - Hardware Failures
- **Fault Tolerance:**
  - *"Enables our system to remain operational and available to the users despite failures within one or multiple of its components."*
- **Tactics for achieving Fault Tolerance**
  - Failure Prevention
    - Redundancy and Replication



  - Failure Detection and Isolation:
    - Monitoring



  - Recovery
    - Stop sending traffic
    - Restart the host
    - Rollback

© 2025 Top Developer Academy LLC by Michael Pogrebinsky. All rights reserved.

Notes:

© 2025 Top Developer Academy LLC by Michael Pogrebinsky. All rights reserved.

SLA, SLO, SLI

- **SLA - Service Level Agreement**
  - It is a legal contract that represents our quality service
- **SLOs - Service Level Objectives**
  - Each SLO represents a target value/range that our service needs to meet
- **SLIs - Service Level Indicators**
  - A quantitative measure of our compliance with a service-level objective
- **Important Considerations:**
  - We shouldn't take every SLI that we can measure in our system and define an objective associated with it
  - Promising fewer SLOs is better
  - Set realistic goals with a budget for error
  - Create a recovery plan for when the SLIs show that we are not meeting our SLOs

Notes:

© 2025 Top Developer Academy LLC by Michael Pogrebinsky. All rights reserved.

API Design

Introduction to API Design for Software Architects

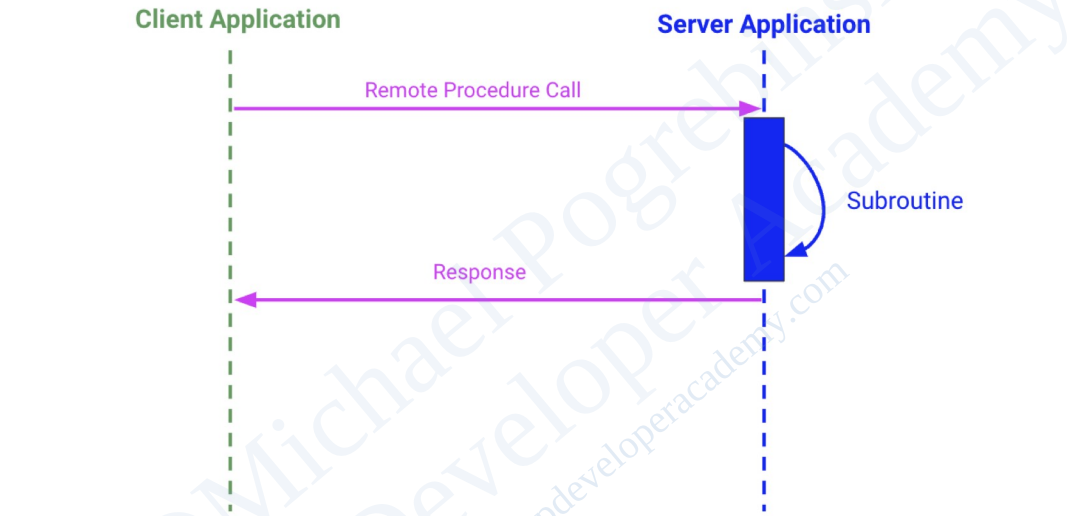
- **An API is a contract between:**
  - Engineers who implement the system
  - Client applications that use the system
- **Categories of API**
  - Public APIs
  - Private/Internal APIs
  - Partner APIs
- **API best practices and patterns:**
  - Complete Encapsulation of the internal design and implementation
  - Easy to Use
  - Keeping the Operations Idempotent
    - *"An operation doesn't have any additional effect on the result if it is performed more than once."*
  - API Pagination
  - Asynchronous Operations
  - Versioning our API

Notes:

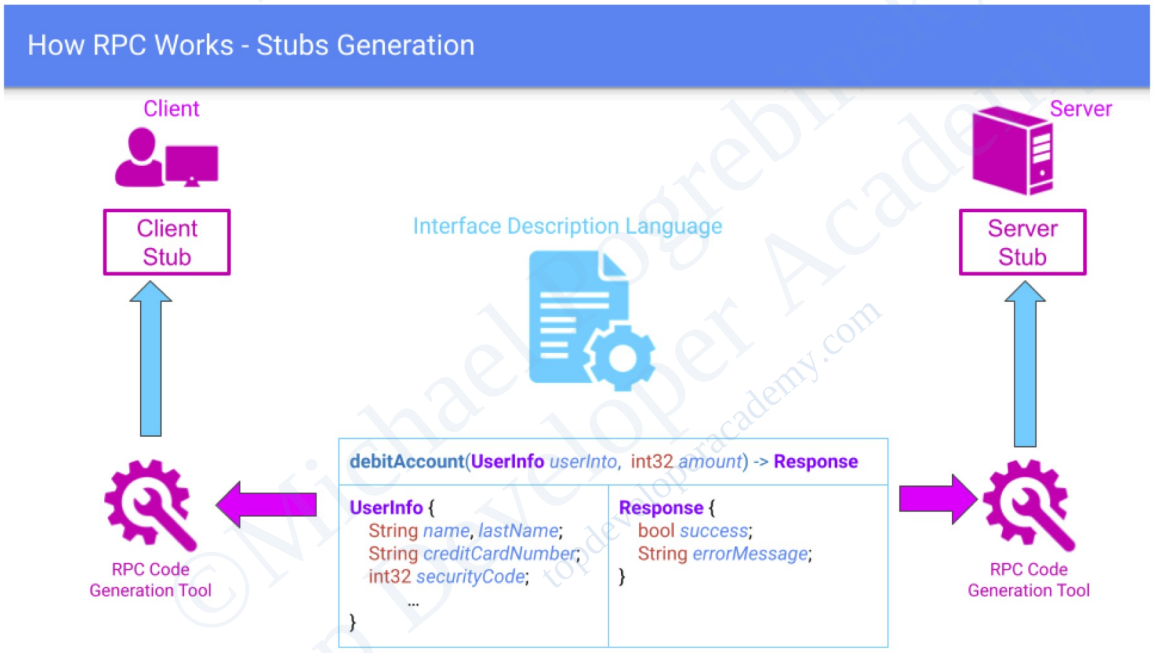
Versioning API - it is necessary because modifying an endpoint may break the usage for our clients

© 2025 Top Developer Academy LLC by Michael Pogrebinsky. All rights reserved.

RPC



- **Features of RPC:**
  - Looks like calling a normal local method
  - RPC frameworks support multiple programming languages



- **Benefits of RPC:**
  - Convenience to the developers
  - The details of communication establishment/data transfer between client to server are abstracted
  - Failures in communication with the server result in an error or exception depending on the programming language

© 2025 Top Developer Academy LLC by Michael Pogrebinsky. All rights reserved.

- **Drawbacks of RPC over local method invocation:**
  - Slower
  - Less reliable

Notes:

RPC API - Remote Procedure Calls

If we just need CRUD operations from our API, then a REST approach may be better

© 2025 Top Developer Academy LLC by Michael Pogrebinsky. All rights reserved.

REST API

- **REST - Representational State Transfer**
  - Set of architectural constraints and best practices for defining APIs for the web
- **Important Concepts:**
  - HATEOAS -
    - The interface is dynamic through Hypermedia as the Engine of the Application State (HATEOAS)
  - Statelessness
  - Cacheability
  - Named Resources - Each resource is either:
    - Simple resource
    - Collection resource
- **Resources - Best Practices:**
  - Naming our resources using nouns
  - Making a distinction between collection resources and simple resources
  - Giving the resources clear and meaningful names
  - The resource identifiers should be unique and URL friendly
- **REST API Operations Mapping to HTTP Methods**
  - REST operations are mapped to HTTP methods as follows:
    - ***Create*** *a new resource* -> **POST**
    - ***Update*** *an existing resource* -> **PUT**
    - ***Delete*** *an existing resource* -> **DELETE**
    - ***Get*** *the state of a resource* } **GET**
    - ***List*** *the sub-resources of a collection* } **GET**
  - In some situations, we define additional custom methods
- **REST API - Step-by-Step Process**
  - Identifying Entities
  - Mapping Entities to URIs
  - Defining Resources' Representations
  - Assigning HTTP Methods To Operations on Resources

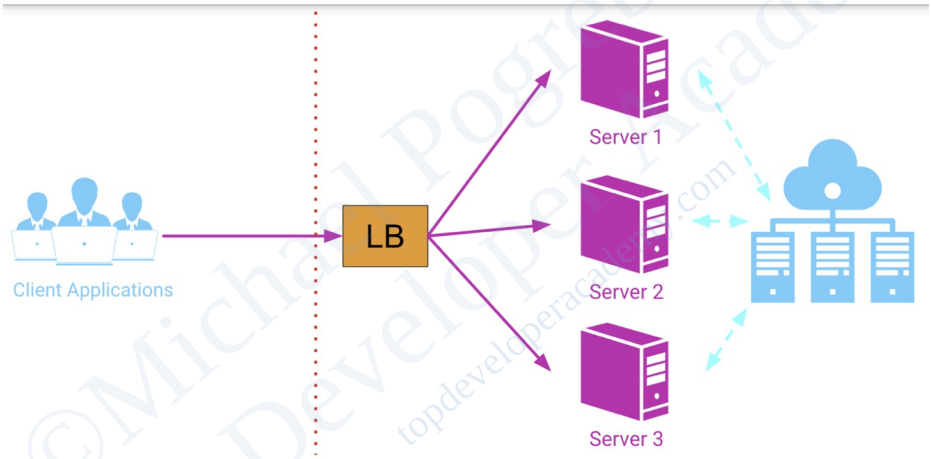
Notes:

© 2025 Top Developer Academy LLC by Michael Pogrebinsky. All rights reserved.

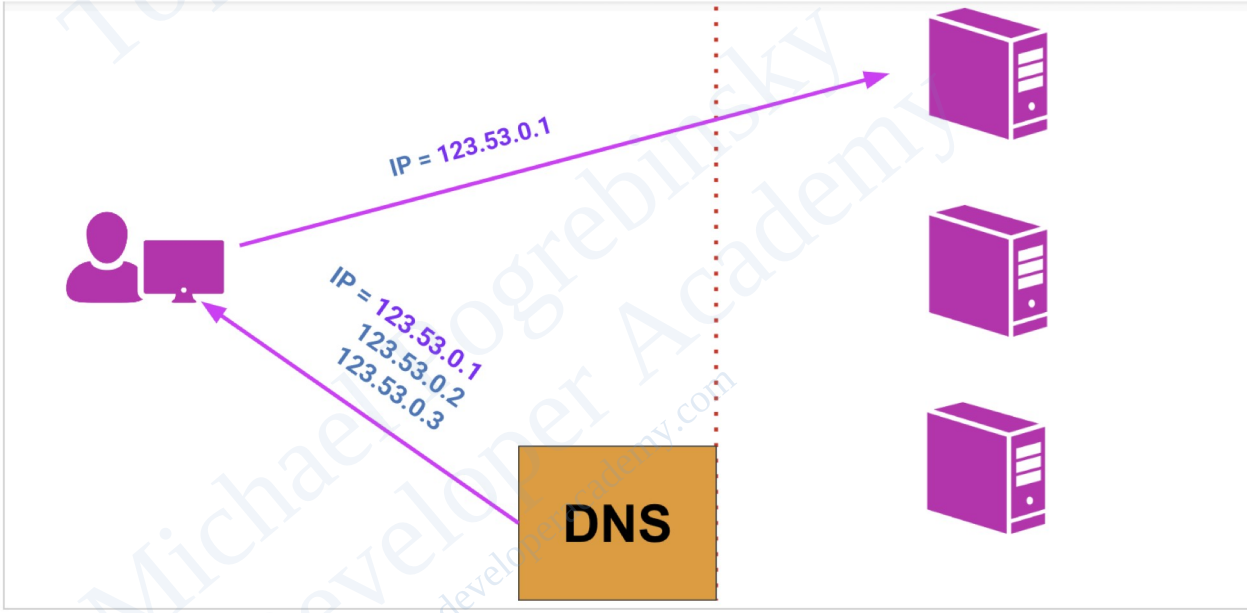
Large Scale Systems Architectural Building Blocks

DNS, Load Balancing & GSLB

- **Role of Load Balancer:**
  - Balance load among a group of servers

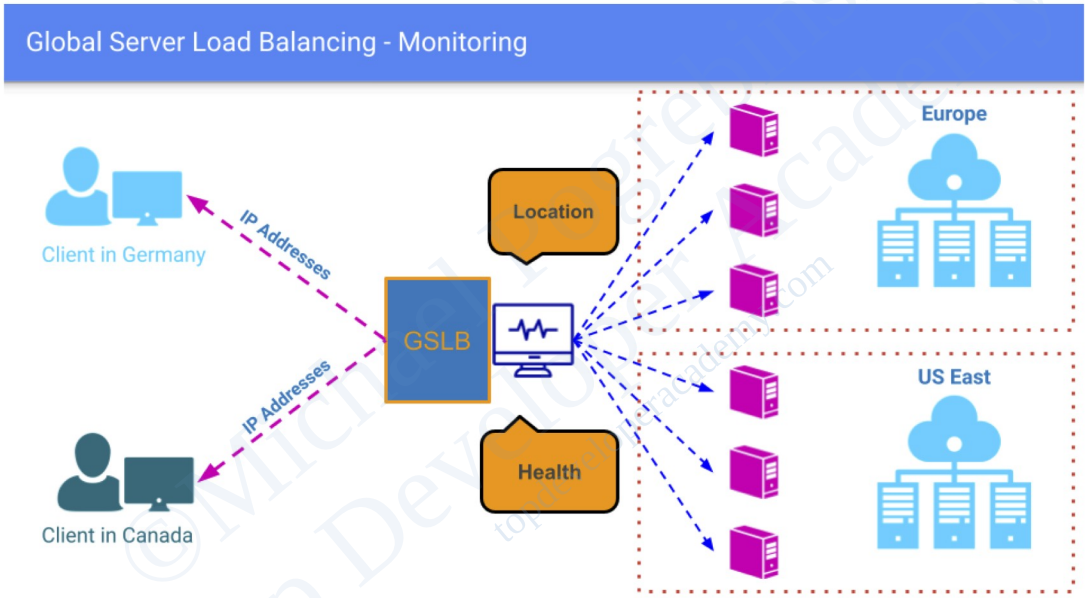


- **Types of load balancers**
  - DNS load balancing



  - Hardware load balancing
    - Run on dedicated devices designed and optimized specifically for load balancing
  - Software load balancing
    - Programs that can run on a general-purpose computer and perform a load-balancing function
  - Global Server Load Balancing

© 2025 Top Developer Academy LLC by Michael Pogrebinsky. All rights reserved.



Notes:

Quality attributes a load balancer provides:

- Scalability -> creating more instances depending on the load

- High Availability

- Performance (Throughput) - it adds latency, but increases the throughput performance

- Maintainability -> we can take servers down one by one and do maintenance without disruption to the clients

DNS load balancer has issues -> that's why we use Hardware/Software load balancer
Global Server Load Balancing is a hybrid between DNS load balancer and Hardware/Software load balancer
It is required because we still need to take into consideration the DNS -> based on the DNS address it redirects to which load balancer (server) the request should go

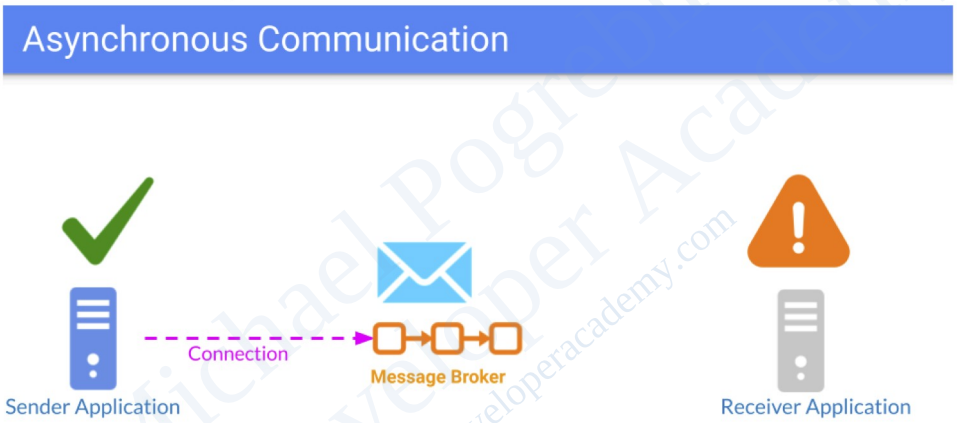
Load Balancing Solutions:
- Open Source: HAProxy, NGINX

- Cloud based: AWS ELB, GCP - Cloud load balancing, Microsoft Azure load balancer

© 2025 Top Developer Academy LLC by Michael Pogrebinsky. All rights reserved.

Message Brokers

- **Definition:**
  - A software architectural building block that uses the queue data structure to store messages between senders and receivers
  - Used inside our system and not exposed externally



- **Benefits:**
  - Services can
    - Publish messages to a particular channel
    - Subscribe to that channel
    - Get notified when a new event is published

Notes:

Message Brokers solutions:
- Open Source: Apache Kafka, RabbitMQ
- Cloud: Amazon Simple Queue Service (SQS), GCP pub/sub and cloud task, Microsoft Azure Service Bus / Event Hubs / Event Grid

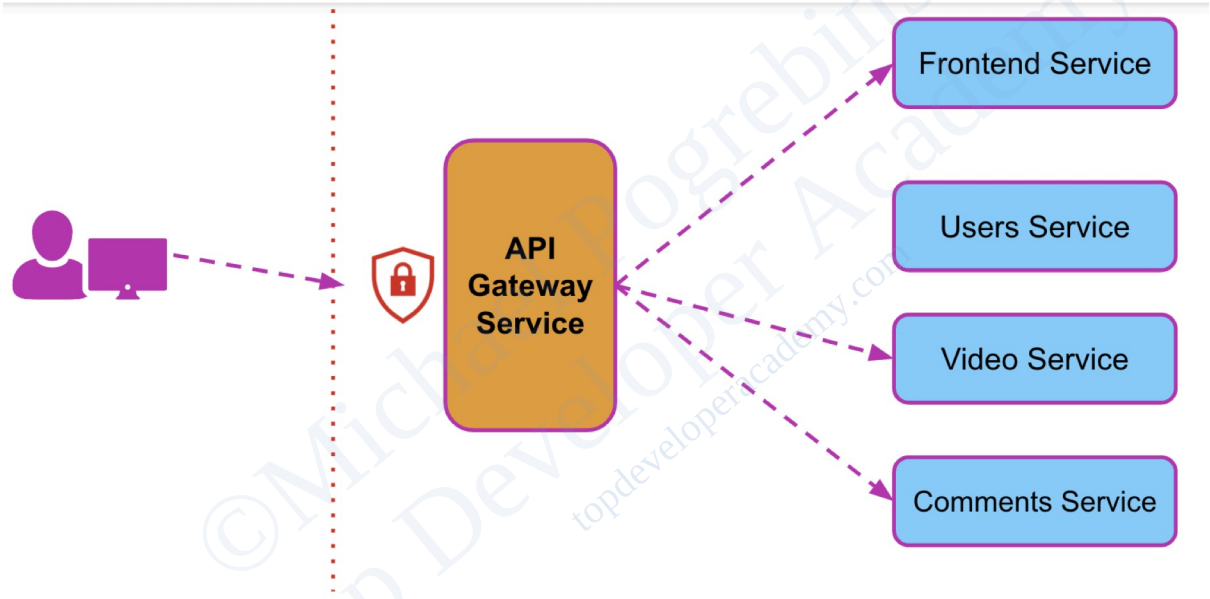
The normal way of communication is Synchronous -> it means that both applications need to be healthy (up and running) at the same time.
Message brokers help us fix this issue and enable the communication even when 2 applications are not healthy at the same time
!!! Message Brokers are used just inside our app, not externally by the user !!!

Quality Attributes:
- Fault Tolerance
- Availability and Scalability
- Performance penalty - it has more latency

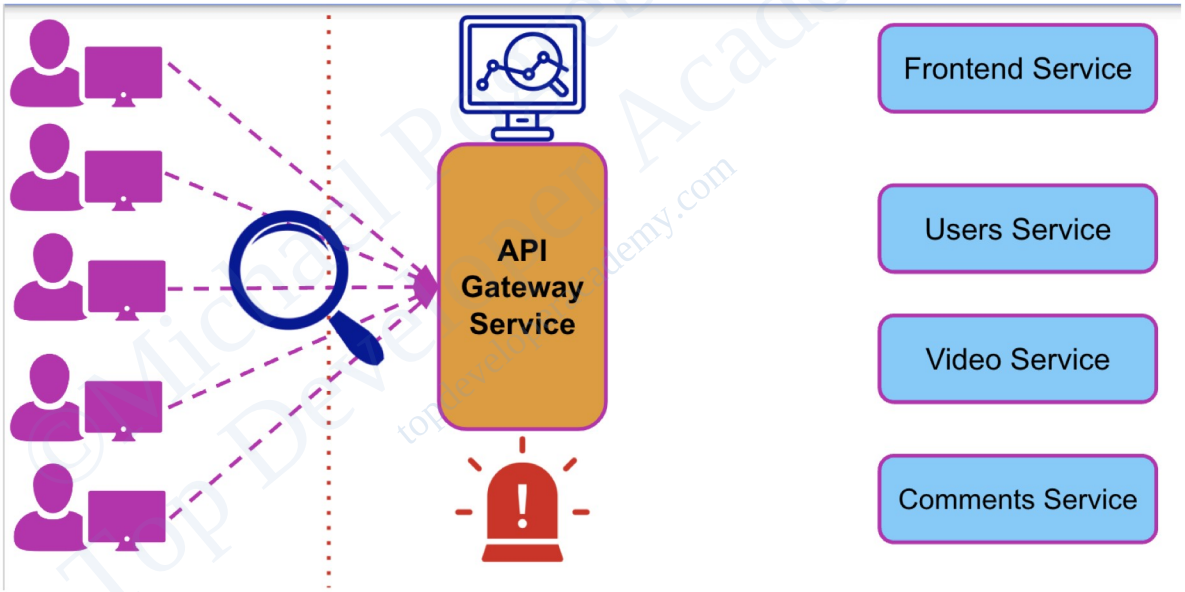
© 2025 Top Developer Academy LLC by Michael Pogrebinsky. All rights reserved.

API Gateway

- **Definition**:
  - Follows a software architecture pattern called “API composition.”
  - The client applications can call one single service

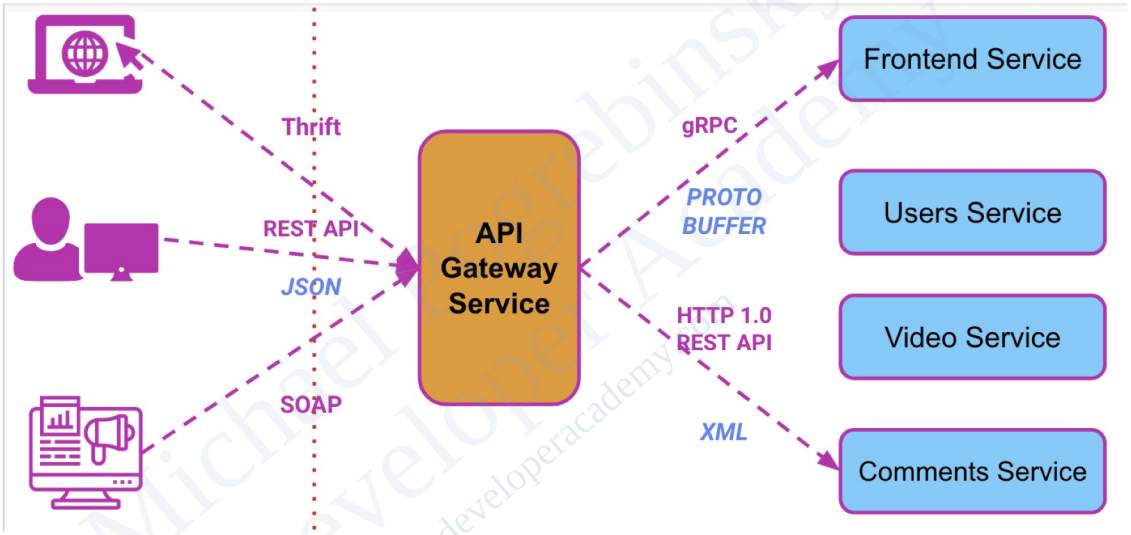


- **Benefits**
  - Seamless internal modifications/Refactoring
  - Consolidating all security, authorization, and authentication in a single place
    Request Routing
  - Static content and response caching
  - Monitoring and alerting



  - Protocol Translation

© 2025 Top Developer Academy LLC by Michael Pogrebinsky. All rights reserved.



- **Considerations**:
  - API Gateway shouldn't contain any business logic
  - API Gateway may become a Single Point of Failure
  - Avoid bypassing API Gateway from external services

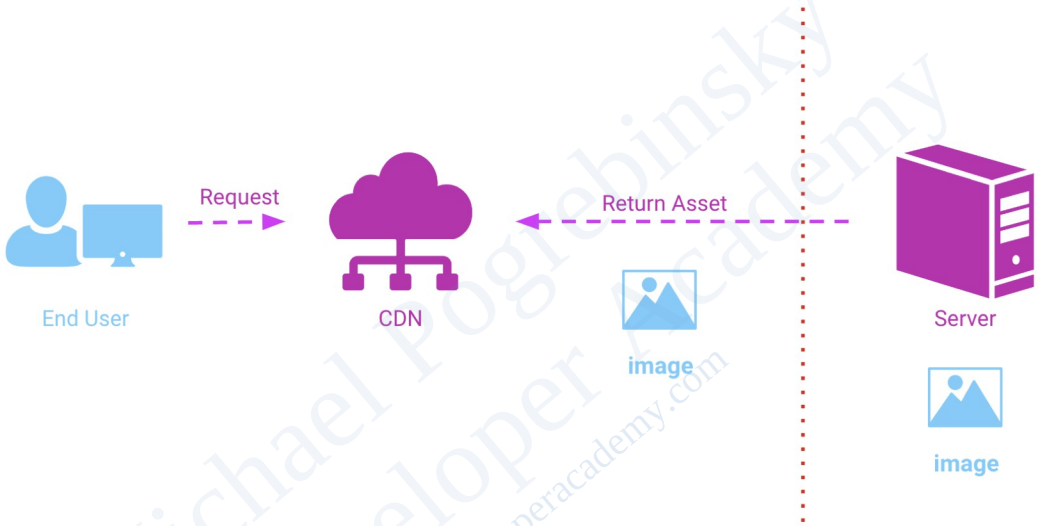
Notes:

API Gateway solutions:
- Open Source: Netflix Zuul, Kong Gateway
- Cloud: Amazon API Gateway, GCP API Gateway, Microsoft Azure API Management

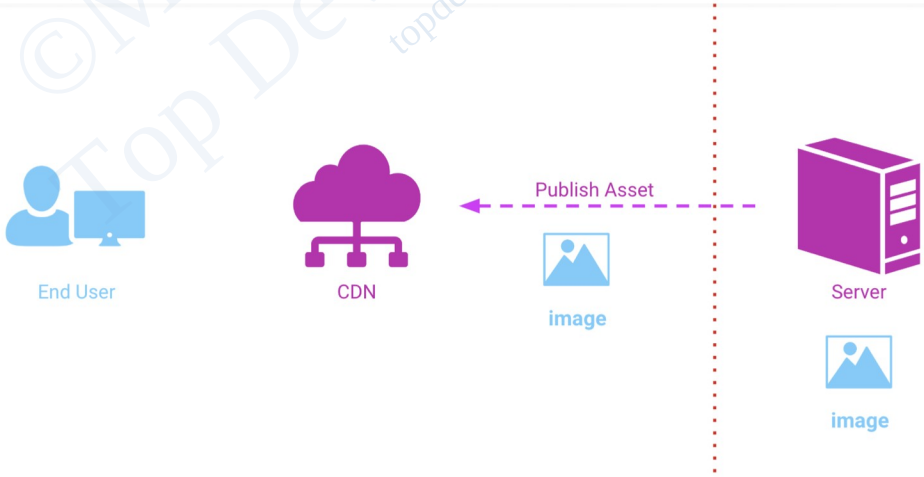
© 2025 Top Developer Academy LLC by Michael Pogrebinsky. All rights reserved.

Content Delivery Network - CDN

- **Definition:**
  - A globally distributed network of servers located in strategic places
- **Main purpose:**
  - Speeding up the delivery of content to end-users
- **Content Publishing Strategies**
  - *Pull Strategy*



  - *Push Strategy*



©Michael Pogrebinsky Top Developer Academy topdeveloperacademy.com

Notes:

CDN Solutions & Cloud Technologies: Cloudflare, Fastly, Akamai, Amazon CloudFront, GCP CDN, Microsoft Azure Content Delivery Network

Quality attributes:
- Performance - faster page loads
- High Availability - issues/slowness are less noticeable
- Security - Protection against DDoS

© 2025 Top Developer Academy LLC by Michael Pogrebinsky. All rights reserved.

Data Storage at Global Scale

Relational Databases & ACID Transactions

- **Properties:**
  - The structure (schema) of each table is defined ahead of time
  - Gives us the knowledge of each what each record must have
- **Advantages:**
  - Ability to form complex and flexible queries
  - Efficient storage
  - Natural structure of data for humans
  - ACID transactions
    - *Atomicity* - Each set of operations that are part of one transaction either:
      - Appear all at once
      - Don't appear at all
    - *Consistency* -
      - A transaction that was already committed is seen by all future queries/transactions
      - A transaction doesn't violate any constraints that we set for our data
    - *Isolation*
      - Related to Atomicity in the context of concurrent operations performed on our database
    - *Durability*
      - Once a transaction is complete, its final state will persist and remain permanently inside the database

©Michael Pogrebinsky Top Developer Academy topdeveloperacademy.com

Notes:

Disadvantages of Relational Databases:

- Rigid structure - all rows inside a table must follow the same schema and the schema needs to be designed ahead of time

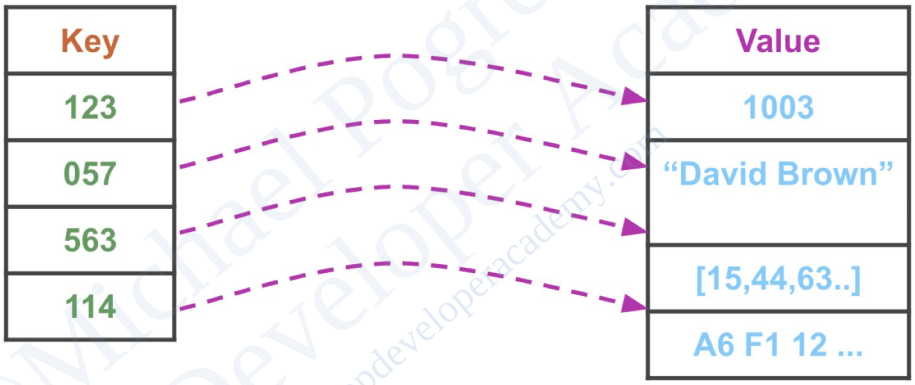
- Harder to maintain/scale

- Slower read operations

© 2025 Top Developer Academy LLC by Michael Pogrebinsky. All rights reserved.

Non-Relational Databases

- Categories:
  - **Key/Value Store**



  - **Document Store**
    - We can store collections of documents with more structure inside each document
    - Each document is an object with different attributes
  - **Graph Database**
    - Optimized for navigating and analyzing relationships between different records

Notes:

Non-Relational Databases solutions:

- Key/Value -> Redis, Aerospike, Amazon DynamoDB

- Document -> Cassandra, MongoDB

- Graph -> Amazon Neptune, NEO4J

© 2025 Top Developer Academy LLC by Michael Pogrebinsky. All rights reserved.

Techniques to Improve Performance, Availability & Scalability Of Databases

- **Database Indexing**

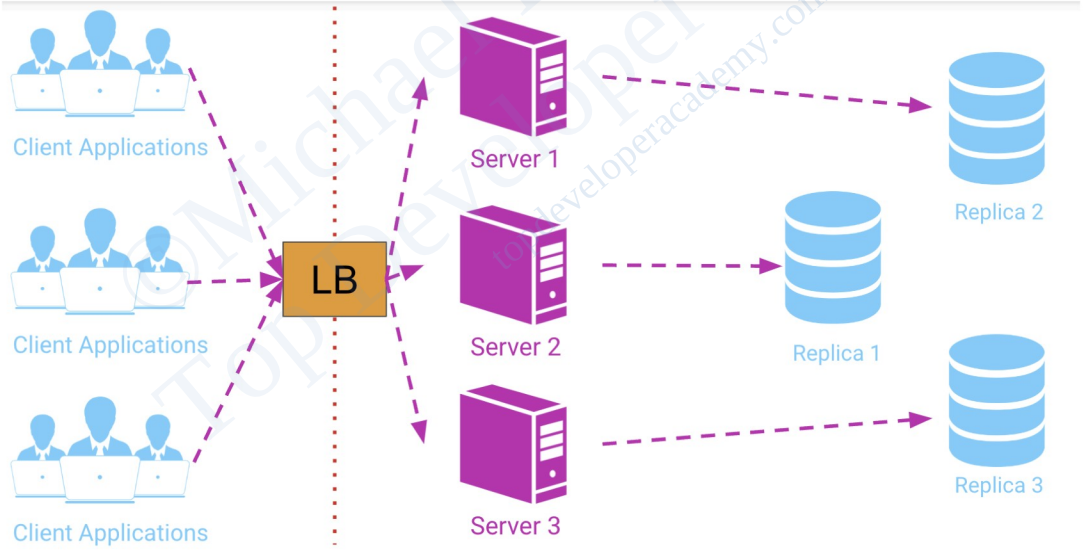
Index Table

City	Row
Los Angeles	1
Boston	2
Los Angeles	3
New York	4
Amsterdam	5
...	...

Users Table

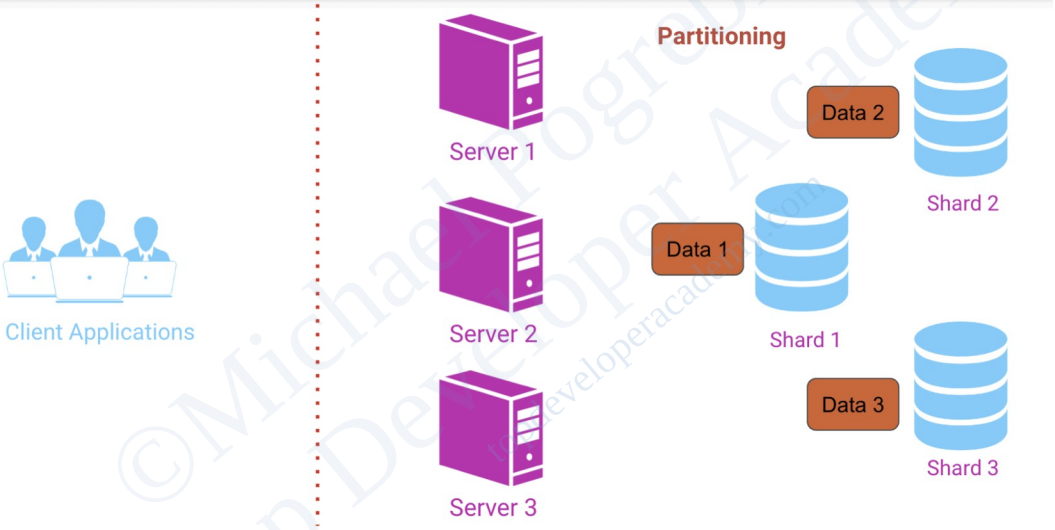
Id	City	FirstName	LastName	Age
1	Los Angeles	Alexander	Ivanov	35
2	Boston	John	Smith	44
3	Los Angeles	Mary	Johnson	15
4	New York	Michael	Bird	86
5	Amsterdam	Joseph	Young	51
...	...	...	...	...

- **Database Replication**



© 2025 Top Developer Academy LLC by Michael Pogrebinsky. All rights reserved.

- **Database Partitioning/Sharding**



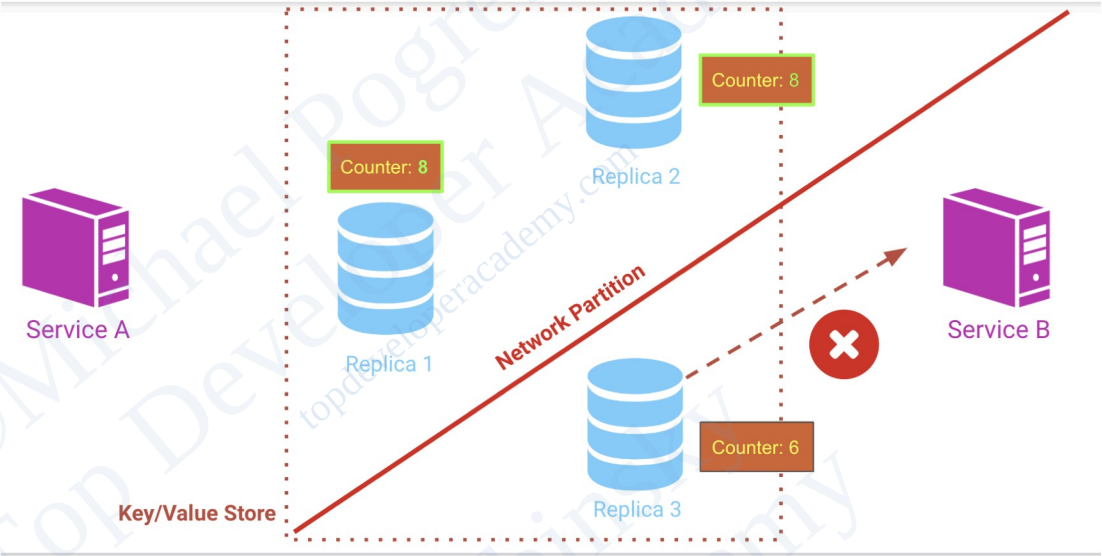
©Michael Pogrebinsky Top Developer Academy

Notes:

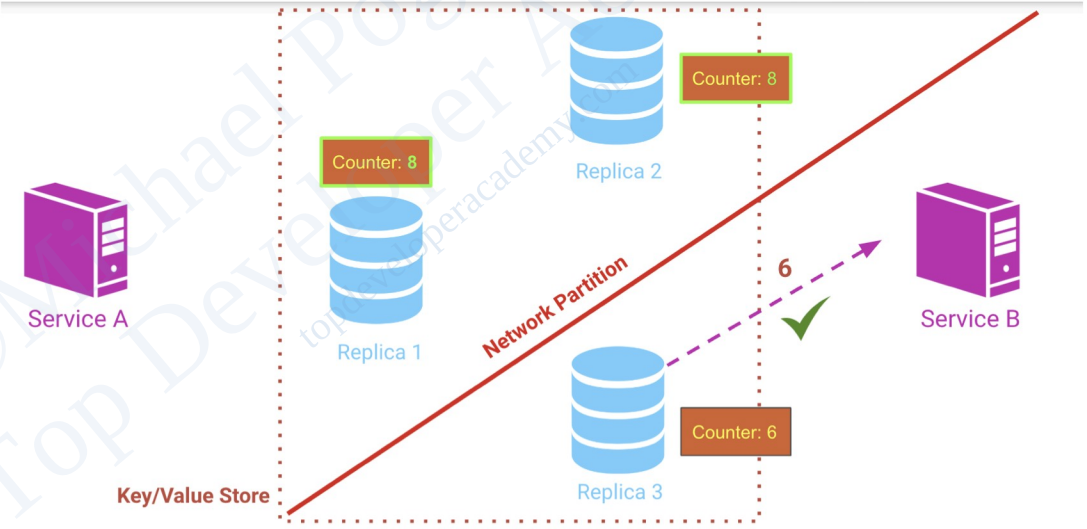
© 2025 Top Developer Academy LLC by Michael Pogrebinsky. All rights reserved.

Brewer's (CAP) Theorem

- Definition:
  - *"In the presence of a Network Partition, a distributed database cannot guarantee both Consistency and Availability and has to choose only one of them."*
- CAP
  - **Consistency**
    - "Every read request receives either the most recent write or an error"



  - **Availability**
    - "Every request receives a non-error response, without the guarantee that it contains the most recent write."



Notes:

The CAP Theorem says we can have only 2 out of 3:
- CA - no Partition Tolerance
- CP - no Availability
- AP - no Consistency

© 2025 Top Developer Academy LLC by Michael Pogrebinsky. All rights reserved.

Unstructured Data Storage

- **Definitions:**
  - Unstructured Data: “Data that doesn’t follow a particular structure, schema, or model.”
  - Blob: *Binary Large Object*
- **Solutions:**
  - DFS - Distributed File System
  - Object Store
    - Object fields:
      - Unique name / Identifier
      - Value - Content
      - Metadata
      - ACL - Access Control List
    - Objects are stored in Containers/Buckets
    - Cloud Solutions are broken into tiers/storage classes:

High Availability/ Performance

Amazon S3	GCP Storage	Azure Blob	Alibaba OSS
S3 Standard	Standard	Hot tier	Standard
Standard - Infrequent Access	Nearline		IA
Glacier Instant Retrieval	Coldline	Cool tier	Archive
Glacier Deep Archive	Archive	Archive Tier	Cold Archive

Limited Access/ Low Performance

Expensive

Cheap

Notes:

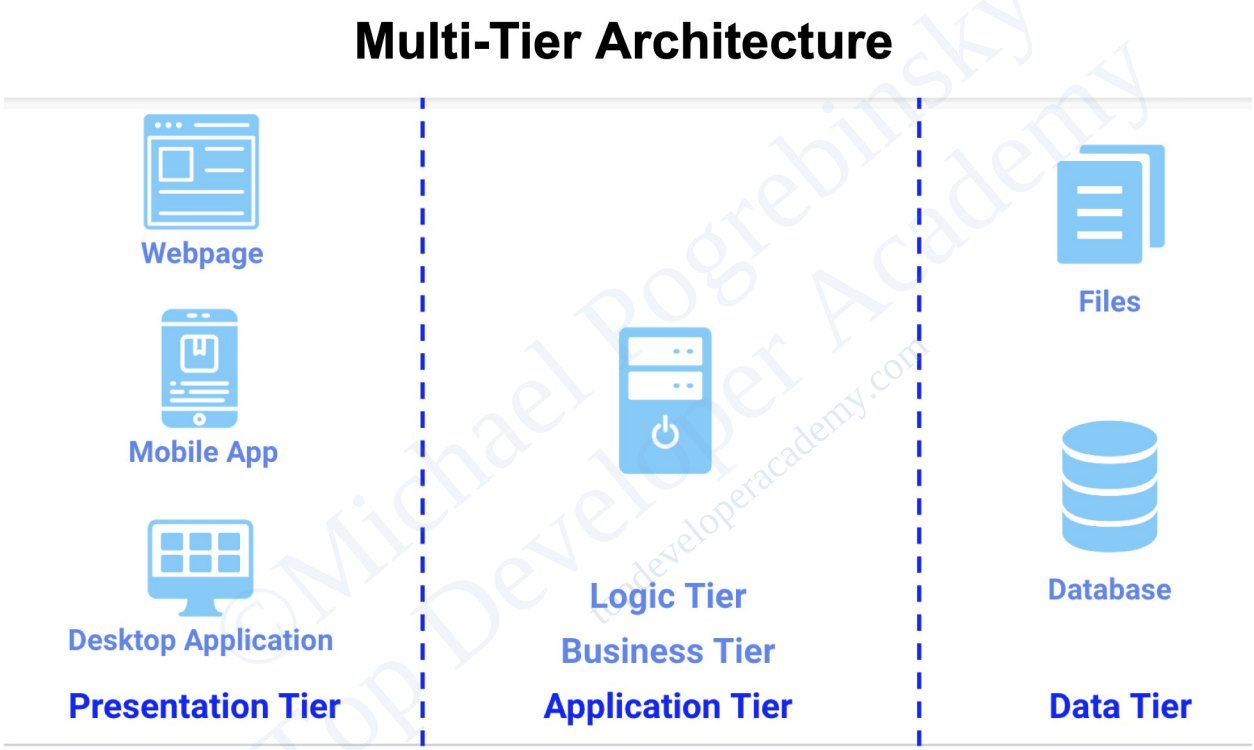
Solutions:
- Cloud Based: Amazon S3, GCP Cloud Storage, Azure Blob Storage, Alibaba Cloud OSS
- Open Source: OpenIO, MinIO, Ceph

Common use-cases:
- Storage of raw user-uploaded data
- Backup and Archiving
- Web Hosting
- ML and Big Data

Two storage solutions:
- Distributed File System -> Scalable/Available version of the File System
- Object Store -> Not as performance as DFS; Best option for the web content

© 2025 Top Developer Academy LLC by Michael Pogrebinsky. All rights reserved.

Software Architecture Patterns



- **Advantages:**
  - Fits a large variety of use cases
  - Easy to scale horizontally
- **Drawbacks:**
  - Monolithic structure of our logic tier

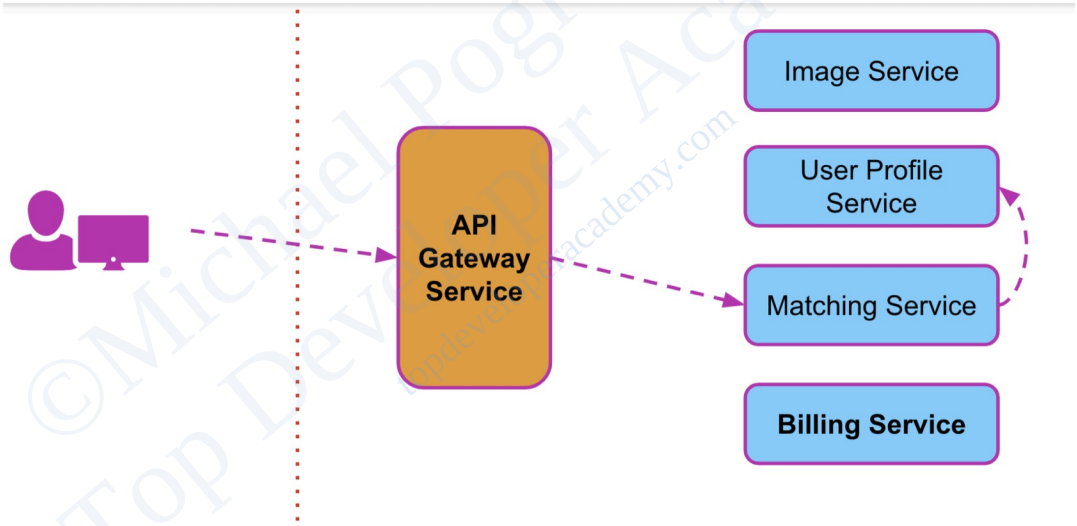
Notes:

Software Architectural patterns are common solutions to software architectural problems that involve multiple components that run as separate runtime units

© 2025 Top Developer Academy LLC by Michael Pogrebinsky. All rights reserved.

Microservices Architecture

- Definition:
  - *"Microservices Architecture organizes our business logic as a collection of loosely coupled and independently deployed services."*
- Best Practices:
  - Single Responsibility Principle
  - Separate Database Per Service



©Michael Pogrebinsky Top Developer Academy

Notes:

Microservice architecture is harder to implement than the 3 tier architecture

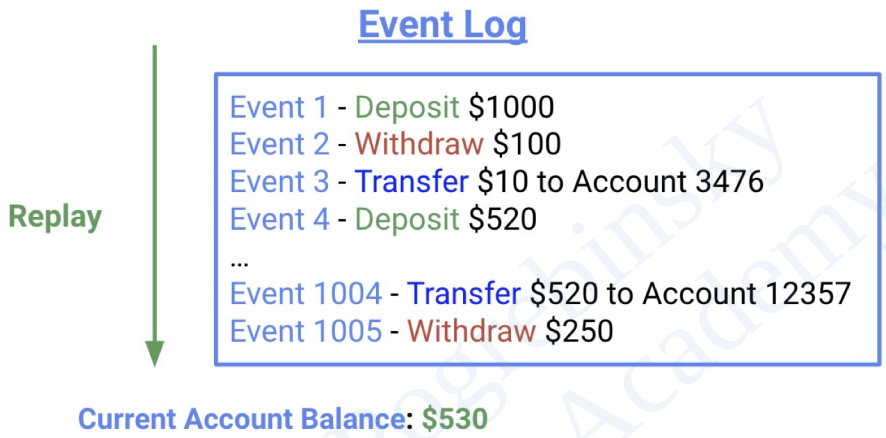
It needs careful consideration and design because the upsides don't come out of the box

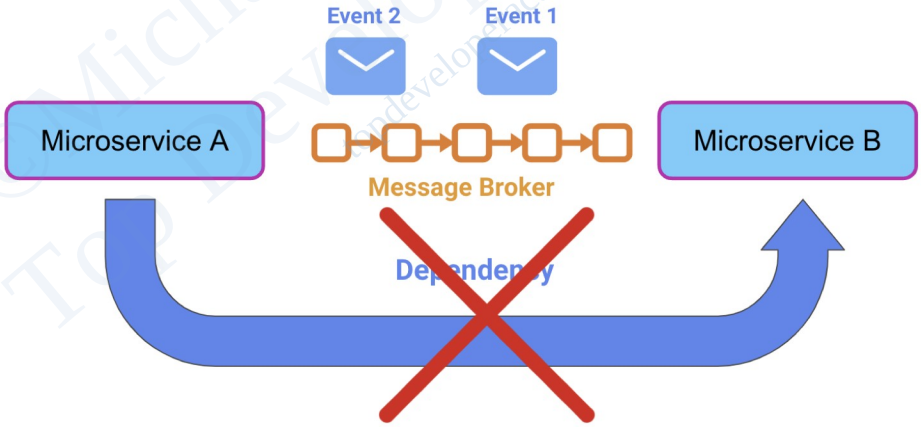
We should consider microservices just when the monolithic approach doesn't work for us

© 2025 Top Developer Academy LLC by Michael Pogrebinsky. All rights reserved.

Event-Driven Architecture

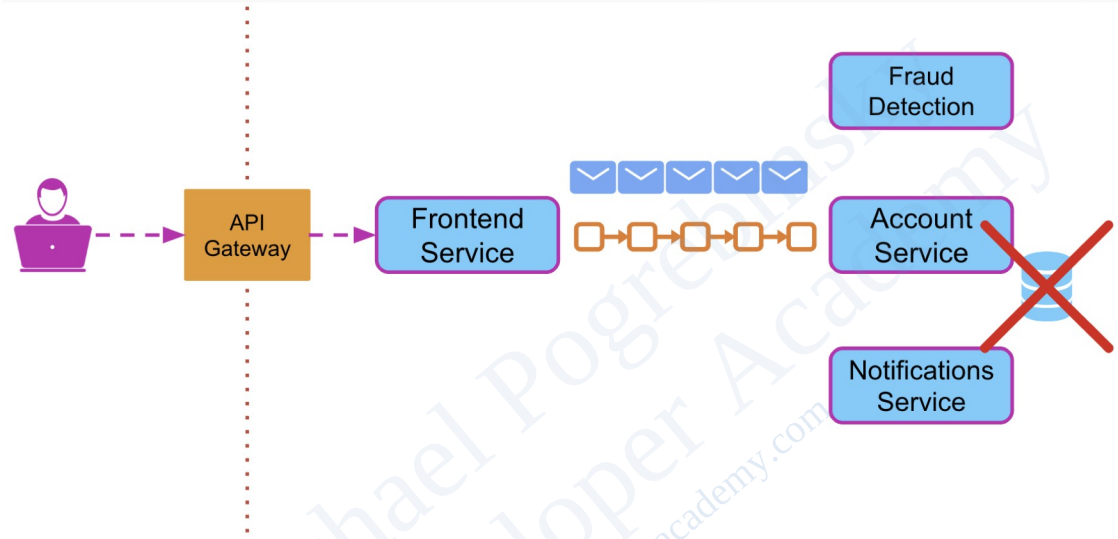
- Definition:
  - An event is an immutable statement of a fact or a change



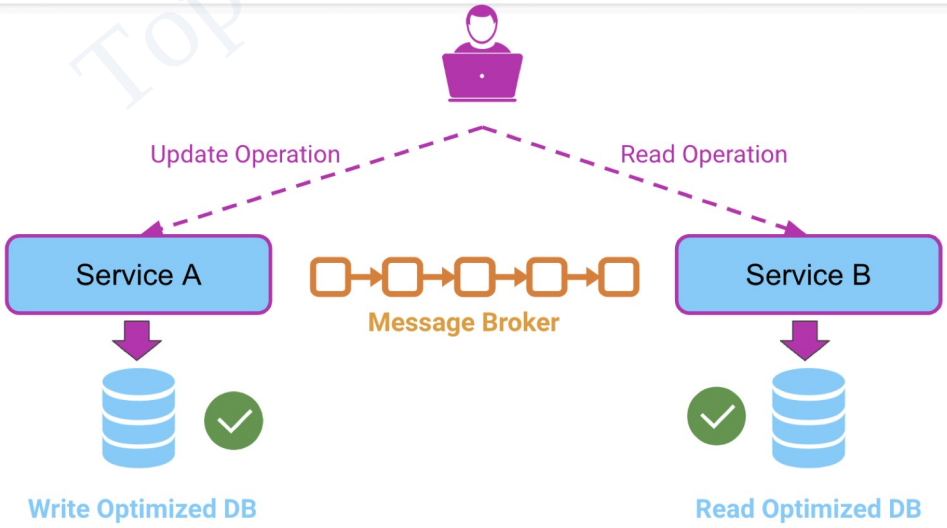


© 2025 Top Developer Academy LLC by Michael Pogrebinsky. All rights reserved.

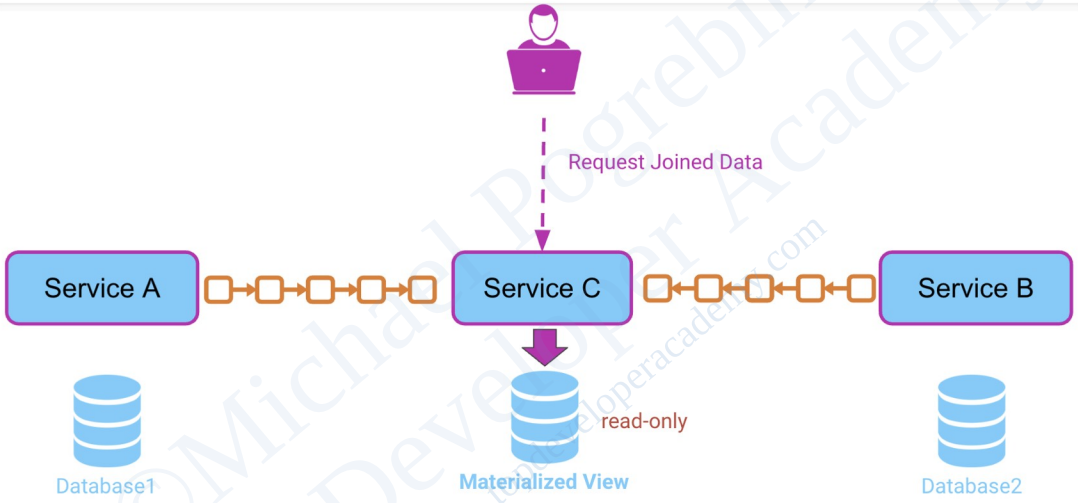
- Event Sourcing Pattern



- CQRS
  - C = Command
  - Q = Query
  - R = Responsibility
  - S = Segregation



© 2025 Top Developer Academy LLC by Michael Pogrebinsky. All rights reserved.



©Michael Pogrebinsky Top Developer Academy
topdeveloperacademy.com

Notes:

The usual microservice architecture has its services tightly coupled with one another because 1 api has to wait for an answer when communicating with another one.

By using event driven architecture this issue doesn't exist anymore

CQRS (Command Query Responsibility Segregation)

CQRS is an architectural pattern that separates operations that change data from operations that read data.

Instead of using the same model for both reading and writing, CQRS introduces two distinct paths:

- Command — changes the system's state.
- Query — retrieves data without changing the system's state.

* Advantages and disadvantages
Advantages

- Independent optimization of reads and writes.
- Cleaner separation of business rules and retrieval logic.
- Better support for complex domain logic.
- Potential for independent scaling and multiple read projections.

Disadvantages

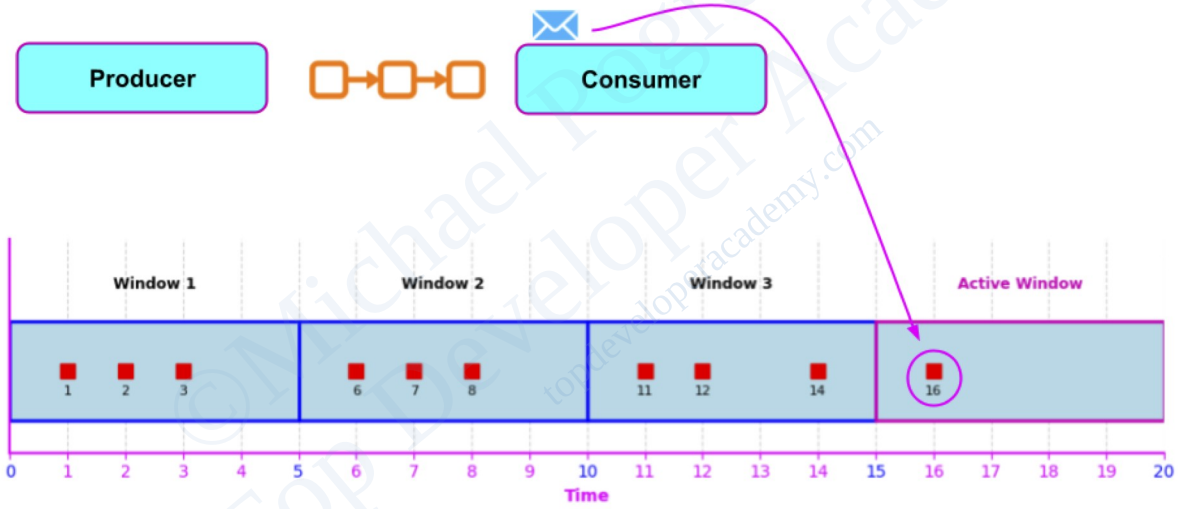
- More code and architectural complexity.
- Read models may be eventually consistent when updated asynchronously.
- More complicated testing, monitoring, and error recovery.
- Additional infrastructure may be needed for asynchronous processing

© 2025 Top Developer Academy LLC by Michael Pogrebinsky. All rights reserved.

Strategies for Processing Infinite Streams of Events

Introduction to Event-Stream Processing and Tumbling Window Strategy

- Processing infinite (unbounded) streams of events requires windowing strategies
- Tumbling Window
  - Fixed sized
  - Back-to-back
  - No overlapping



©Michael Pogrebinsky Top Developer Academy topdeveloperacademy.com

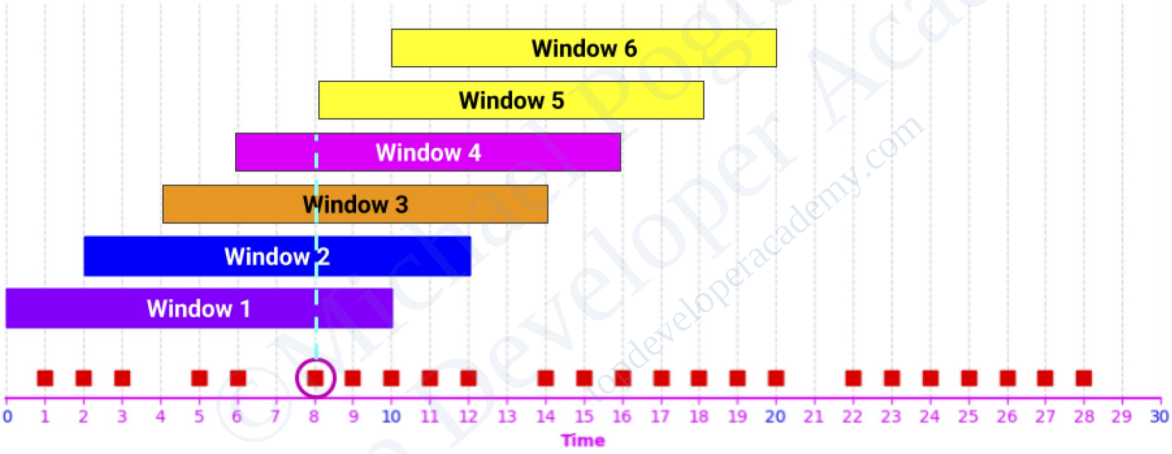
Notes:

Cons:
- the user gets an update just at regular intervals of time, so for example if we have a 24h update window, in the 23rd hour the user will still see the old data
- it's not good for pattern/trend recognition

© 2025 Top Developer Academy LLC by Michael Pogrebinsky. All rights reserved.

Hopping Window Event-Stream Processing Strategy

- Hopping Window Strategy
  - Fixed-size windows
  - Overlapping with an advanced interval
  - Each event potentially belongs to more than one window



Notes:

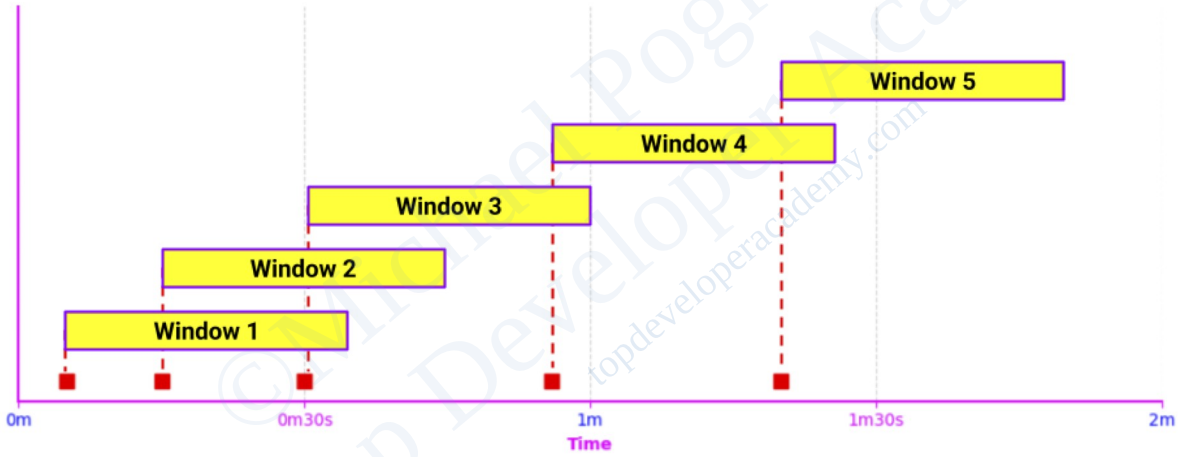
Pros:
- produce results of processing events more frequently
- great for UIs and dashboards monitored by humans
- allows us to respond to changes/insights faster and sooner

Cons:
- the events are aggregated by the "consumer" in memory
- instead of one window at the time we have multiple windows and/or more events in flight -> increase the memory footprint of the consumer microservice
- more frequent aggregation of events -> more CPU resources & more network traffic

© 2025 Top Developer Academy LLC by Michael Pogrebinsky. All rights reserved.

Sliding Window Event-Stream Processing Strategy

- Sliding window strategy
  - Fixed-size windows
  - Dynamic advance interval
    - A new window is created for every new event
  - Each event potentially belongs to multiple windows



©Michael Pogrebinsky Top Developer Academy topdeveloperacademy.com

Notes:

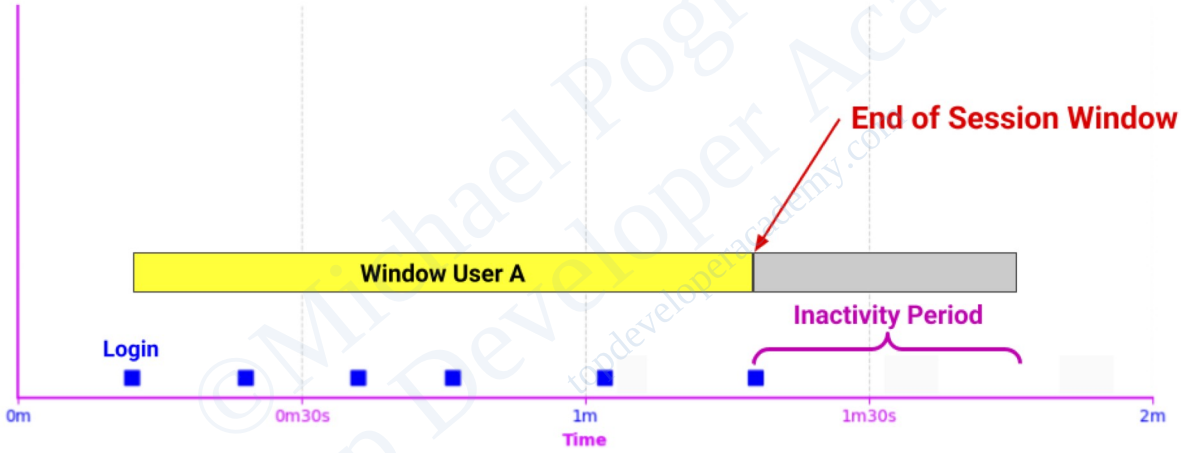
Pros:
- Excellent strategy for real-time monitoring & pattern detection
- Sliding windows vs Hopping windows -> advance interval is flexible

Cons:
- Processing and memory costs are higher -> especially if the volume of events is very high

© 2025 Top Developer Academy LLC by Michael Pogrebinsky. All rights reserved.

Session Window Event-Stream Processing Strategy

- Session Window strategy
  - No global
  - Dynamic window size
  - Opens for specific events
  - Closes after an inactivity threshold is met



©Michael Pogrebinsky Top Developer Academy topdeveloperacademy.com

Notes:

Pros:
- Excellent strategy for real-time analytics for users using App or Service
- Dynamic window size -> allows grouping events based on activity
- Each event belongs to only one window

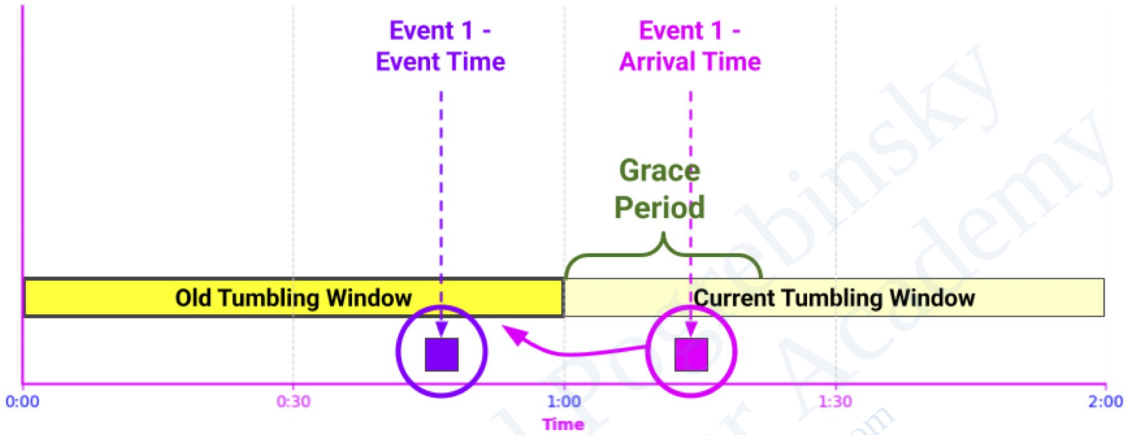
Cons - challenges:
- defining the inactivity threshold/period
- typical situation: a person just stops using the app/browser
- we can't know if the activity ended or not - maybe the user just took a break

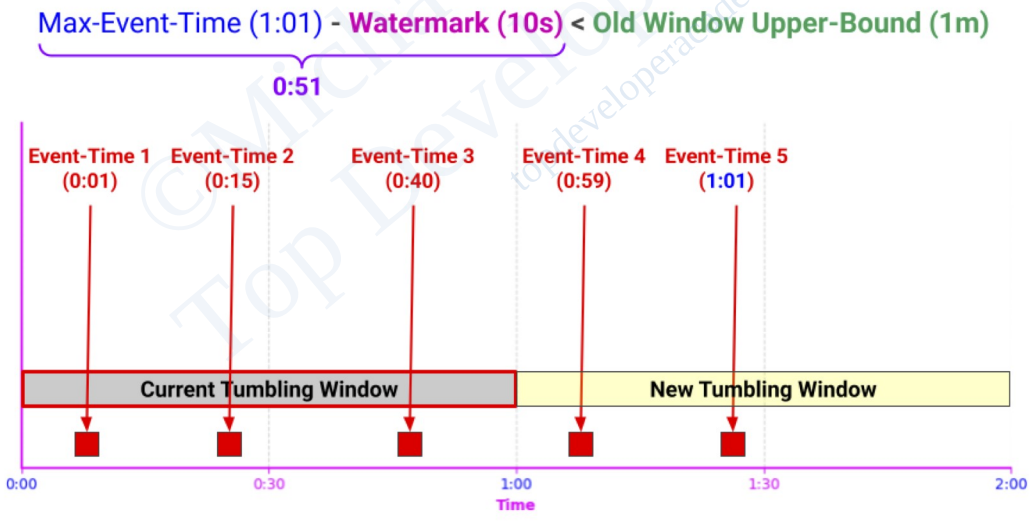
© 2025 Top Developer Academy LLC by Michael Pogrebinsky. All rights reserved.

Time in Stream Processing and Handling Late Arrival of Events

- **Types of timestamps:**
  - Event Time
  - Application Time / Arrival Time
  - Processing Time

- **Options for handling the late arrival of events**
  - Discarding events
  - Grace period
  - Watermarking





© 2025 Top Developer Academy LLC by Michael Pogrebinsky. All rights reserved.

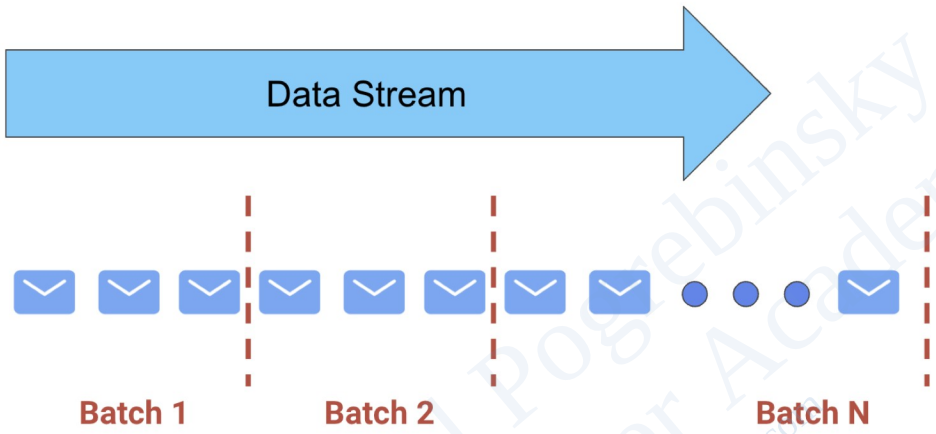
Notes:

© 2025 Top Developer Academy LLC by Michael Pogrebinsky. All rights reserved.

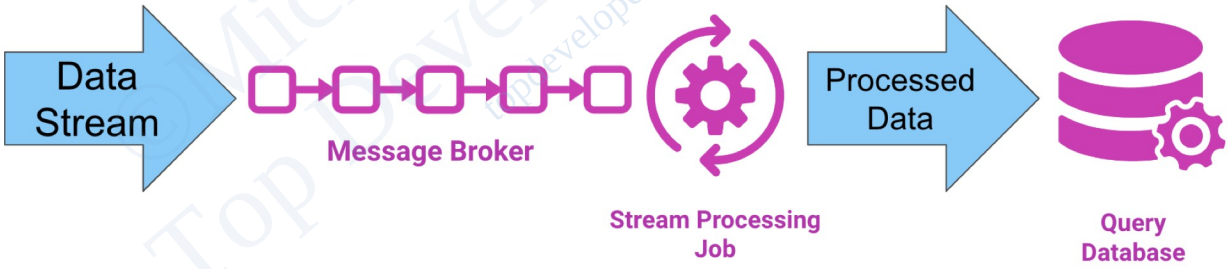
Big Data Architecture Patterns

Big Data Processing Strategies

- **Batch Processing**



- **Real-Time Processing**



Michael Pogrebinsky
Top Developer Academy
@topdeveloperacademy.com

Notes:

Big Data -> datasets that are either
- too large in size
- too complex in structure
- come to our system at a high rate

that exceeds the capacity of a traditional application

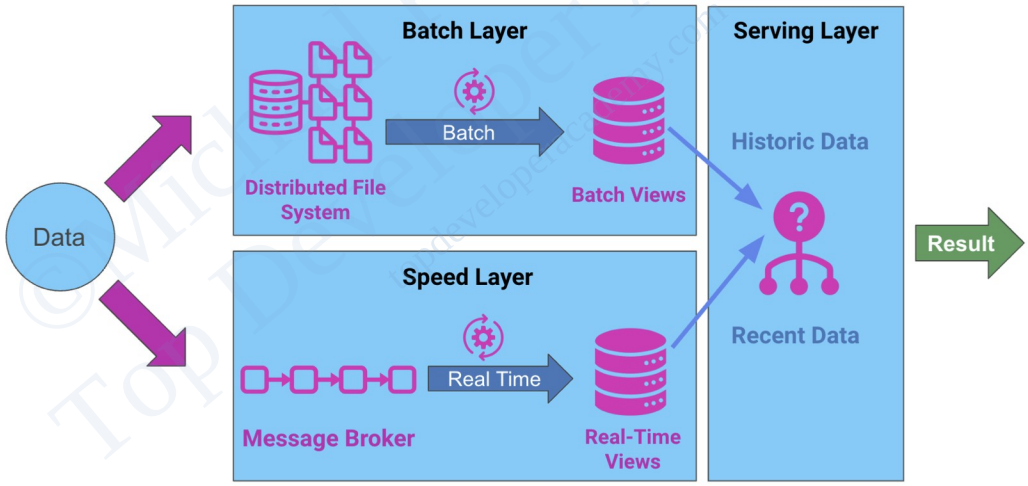
Batch processing -> data is not processed individually, but in fixed batches and it's usually scheduled (ex: once a month)
- advantages: easy to implement, high availability, better efficiency, higher fault tolerance
- drawbacks: long delay between incoming data and processing job

Real-time processing
- advantages: we can analyze and respond to data as it comes immediately
- cons: it's hard to do any complex analysis in real-time

© 2025 Top Developer Academy LLC by Michael Pogrebinsky. All rights reserved.

Lambda Architecture

- Layers:
  - Batch Layer
  - Speed Layer
  - Serving Layer



Notes:

Whenever we need both batch and real time processing we can go for lambda architecture which takes advantages from both

The batch layer works very similar to the batch architecture

The speed layer is there to close the gap between the incoming data and the batch processing time
- it operates just on the most recent data

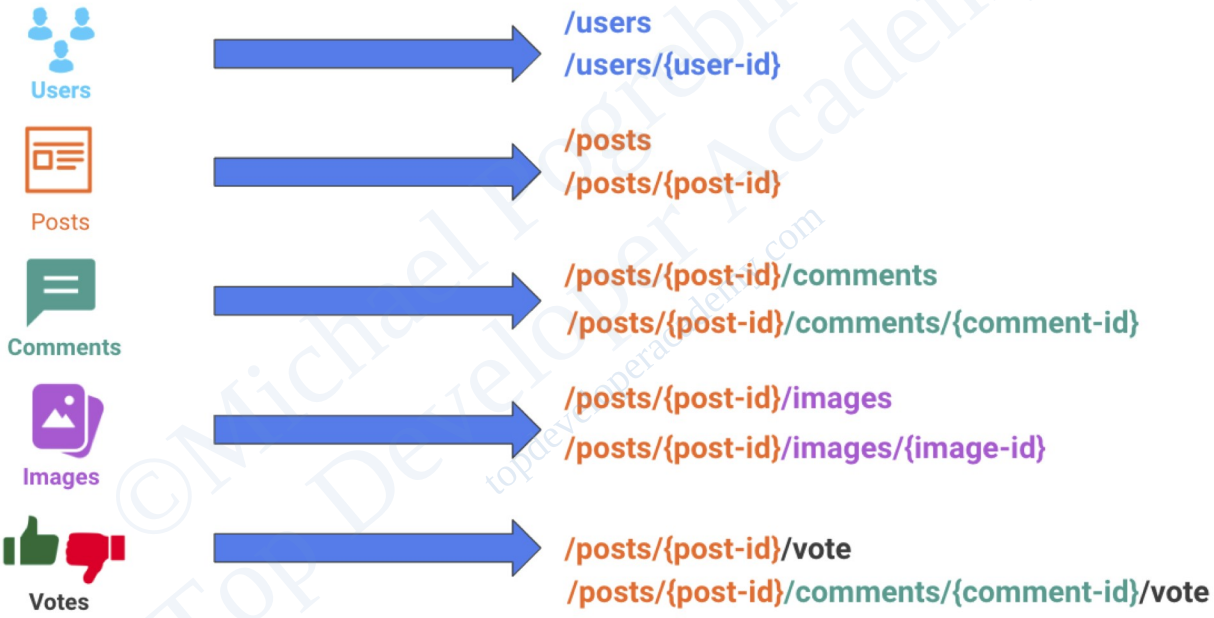
The serving layer merges the data from both layers and queries them

© 2025 Top Developer Academy LLC by Michael Pogrebinsky. All rights reserved.

Software Architecture & System Design Practice

Design a Highly Scalable Discussion Forum 1 - Requirements & API

- **API Definition:**
  - Entity URLs:



©Michael Pogrebinsky Top Developer Academy topdeveloperacademy.com

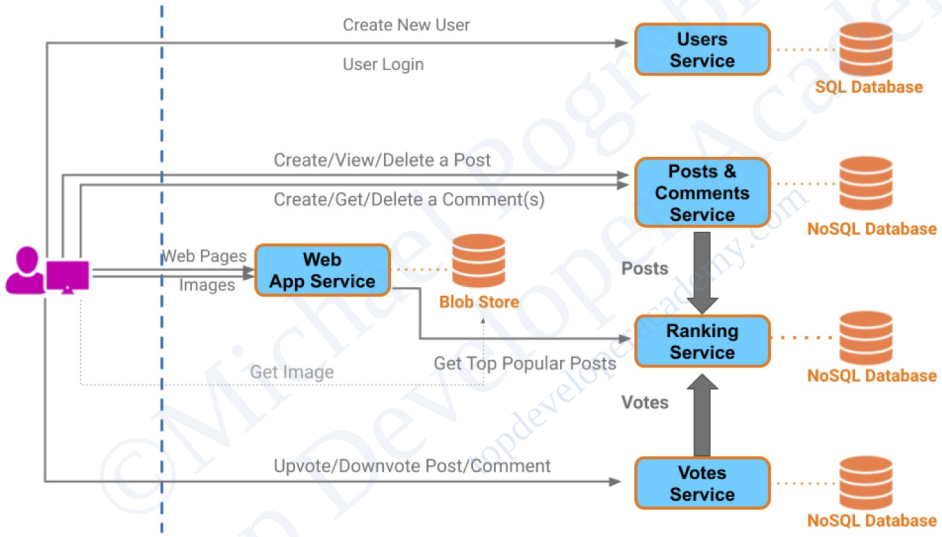
Notes:

Summary:
- started with initial vague / ambiguous requirements
- asked questions
- gathered functional requirements
- gathered / defined the desired qualoty attributes
- REST API definition steps:
> identified entities
> mapped entities to URIs
> defined the representation of our resources
> assigned operations to HTTP methods
- API Observations:
> API pagination

© 2025 Top Developer Academy LLC by Michael Pogrebinsky. All rights reserved.

Design a Highly Scalable Discussion Forum 2 - Functional Architecture Diagram

- **Functional Software Architecture Diagram:**

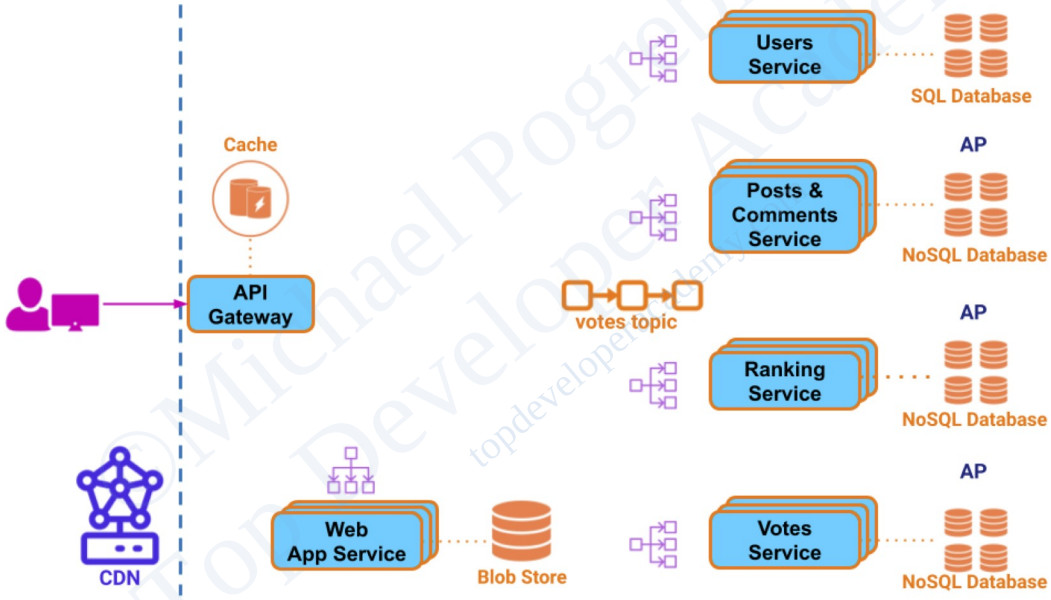


Notes:

© 2025 Top Developer Academy LLC by Michael Pogrebinsky. All rights reserved.

Design a Highly Scalable Discussion Forum 3 - Final Software Architecture

- **Final Software Architecture Diagram:**



Summary:

- Scalability
> Load Balancing
> Database sharding
> API Gateway

- Performance
> CDN
> Caching
> Database Indexing

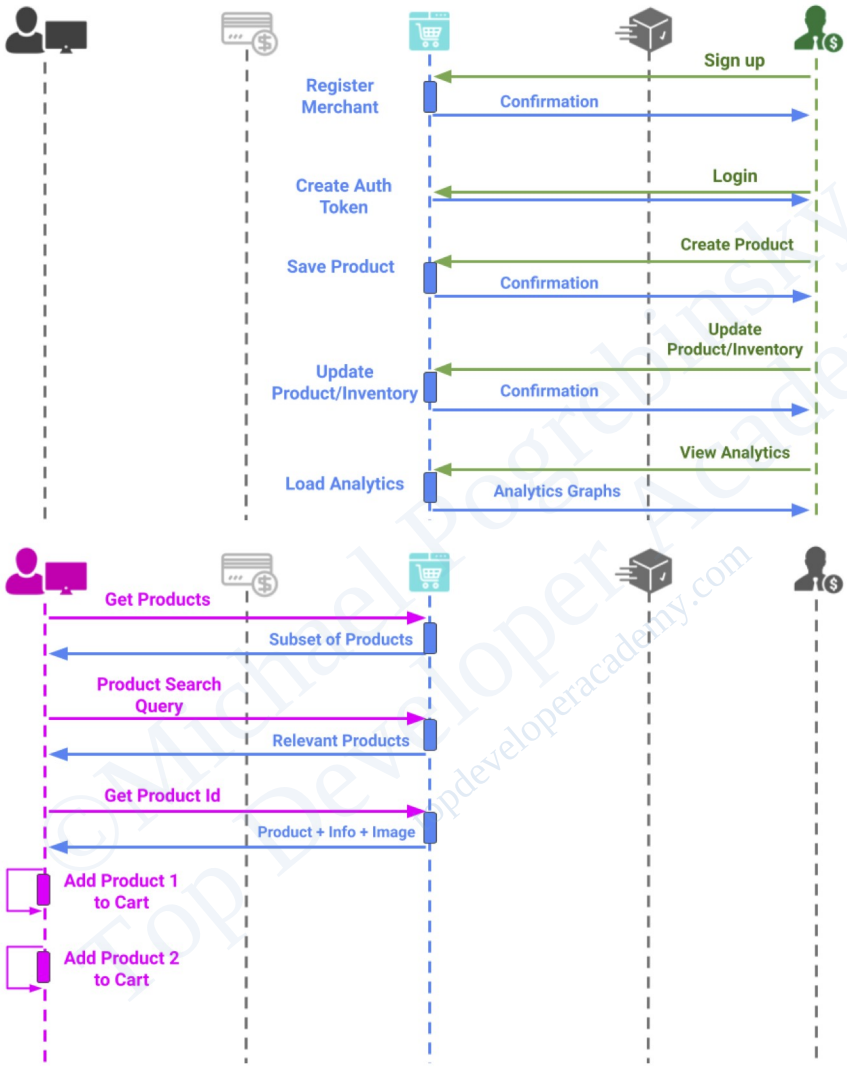
- High Availability
> Database replication
> Redundancy
> Message Broker
> Multi Data Center Deployment

- Availability over Consistency
> Choosing / Configuring AP Databases

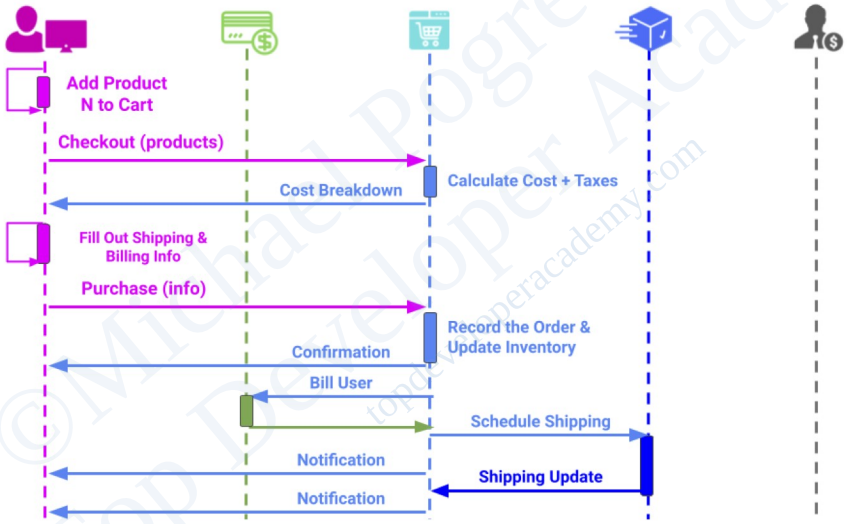
© 2025 Top Developer Academy LLC by Michael Pogrebinsky. All rights reserved.

Design an E-Commerce Marketplace Platform 1 - Requirements & Sequence Diagram

- **Sequence Diagram:**



© 2025 Top Developer Academy LLC by Michael Pogrebinsky. All rights reserved.



Notes:

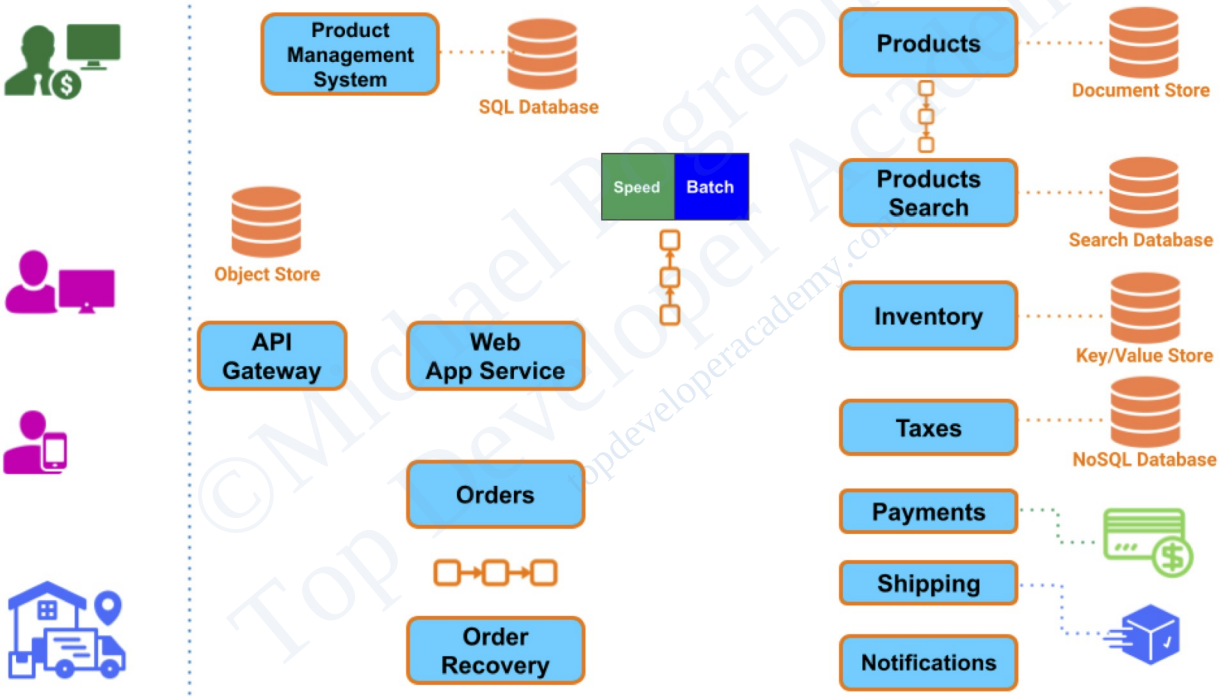
Quality Attributes Requirements - Merchants:
- Scalability (not very high)
> Hundreds of merchants
> Low Traffic
> Thousands of products
- Performance -> response time < 1 seconds at 50th percentile
- Consistency vs Availability -> CP databases
- High Availability -> uptime 99.5%

Quality Attributes Requirements - Users / Buyers:
- Scalability (very high)
> 10-100 Million daily users
> Multiple countries
> High traffic at peak
- Performance
> Product response time < 200ms 50p, 500ms 99p
> Checkout response time < 1 second 99p
- Consistency vs Availability
> Storefront: AP
> Checkout: CP
- High Availability -> Internal SLA -> uptime 99.99%

© 2025 Top Developer Academy LLC by Michael Pogrebinsky. All rights reserved.

Design an E-Commerce Marketplace Platform 2 - Functional Diagram

- **Functional Software Architecture Diagram:**



Notes:

Summary:

- Microservices architecture for organizational scalability

- CQRS to separate a write-intensive CP database from read-intensive AP database

- NoSQL database with a dedicated text search engine

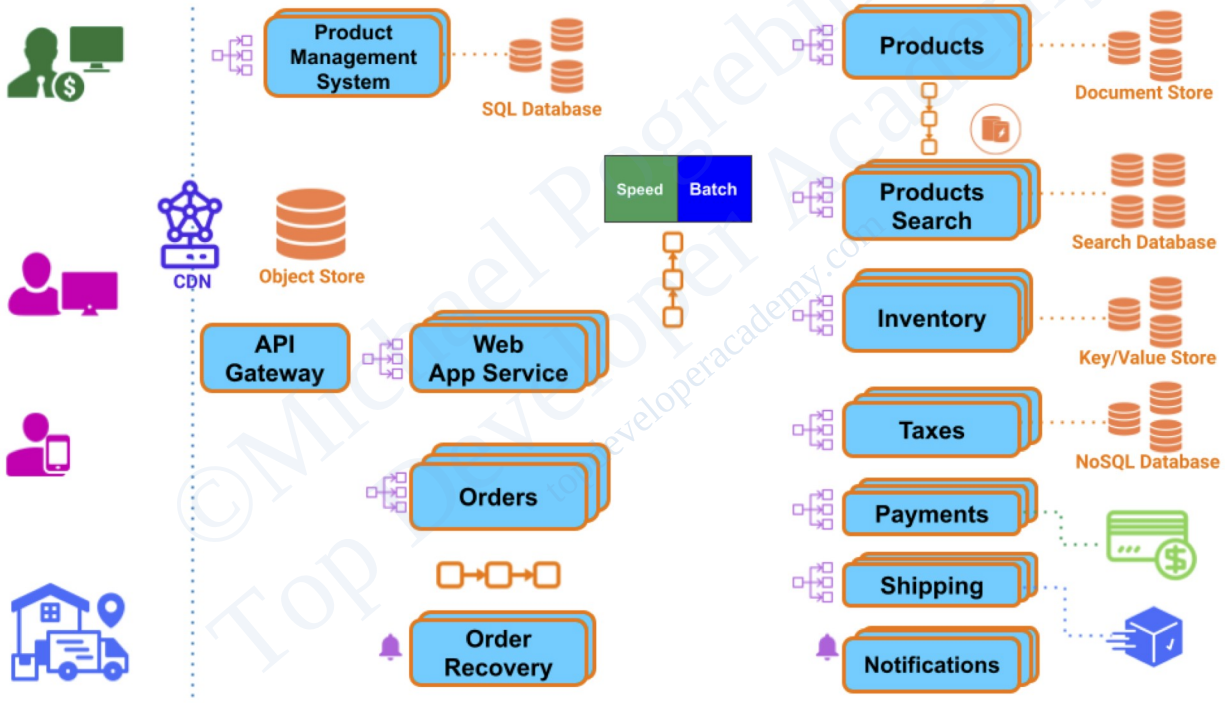
- Tradeoff between synchronous and asynchronous processing

- Event Sourcing for failure recovery, auditing and organizational scalability

- Lambda Architecture to provide real-time and batch processed historic and projected data

© 2025 Top Developer Academy LLC by Michael Pogrebinsky. All rights reserved.

Design an E-Commerce Marketplace Platform 3 - Final Software Architecture



Notes:

Non-functional requirements diagram

© 2025 Top Developer Academy LLC by Michael Pogrebinsky. All rights reserved.